```
/*
================================================================================
=====
   etl-migration-gcp-8-odi-repository-sorgulari-en.sql
   ODI repository discovery script - ETL Migration Assessment (Legacy ETL ->
Google Cloud)

================================================================================
=====

   PURPOSE
     Read-only discovery of an Oracle Data Integrator estate. Rebuilds the two
form sheets
     that were produced from repository queries:
       - Sheet "3 - Pipeline & Job Envanteri"  <- queries 01, 02 (and 08 for
schedules)
       - Sheet "5 - Performans & SLA"          <- query 09 (and 09b for the SLA
window)
     and goes deeper where the inventory had blind spots: mapping complexity and
KMs (03),
     procedure text (04), variables (05), journalizing/CDC (06), non-database
sources (07),
     load plan structure (08), daily volumes (10), target DB dictionary (11),
summaries (12).

   TARGET
     ODI 12c (12.2.1.x) work repository on Oracle Database. In 12c the RCU
usually creates
     one schema (<PREFIX>_ODI_REPO) holding both master and work repository
tables, query
     07c reads master tables (SNP_LSCHEMA, SNP_CONNECT...) from the same schema.
     Query 11 is the exception: it runs on the DWH (target) database, not the
repository.

   READ-ONLY
     Every statement is a SELECT. The only non-SELECT statements are
SQL*Plus/SQLcl client
     commands (DEFINE, SET, SPOOL) and ALTER SESSION SET CURRENT_SCHEMA, which
changes name
     resolution for this session only. Nothing is written to the database. The
user needs
     SELECT on the SNP_ tables (a read-only account with grants is enough). If
access is
     through synonyms in another schema, set CURRENT_SCHEMA to the schema that
owns the
     tables, or remove the ALTER SESSION line and rely on the synonyms (query 00
then needs
     OWNER changed to the owning schema).

   HOW TO RUN
     SQLcl     : sql user@//host:1521/service  then  @etl-migration-gcp-8-odi-
repository-sorgulari-en.sql
                 SET SQLFORMAT CSV is set below, each query spools to its own
CSV file.
     SQL*Plus  : 12.2+ client, replace SET SQLFORMAT CSV with SET MARKUP CSV ON
QUOTE ON.
     SQL Dev.  : Run as script (F5) to get the spool files, or run one statement
at a time
                 (Ctrl+Enter) and export the grid (right click > Export > CSV /
Excel).
     Run 00 first and read its output. Any MISSING column means a later query
must be
     adjusted before it runs (see the "-- VERIFY:" comments and sql_notes.md).
     Queries keep going after an error (WHENEVER SQLERROR CONTINUE), so one
```

```
broken query
     does not stop the rest, check each CSV for ORA- errors.

   PARAMETERS (substitution variables, set once below)
     ODI_WORK_SCHEMA  schema that owns the SNP_ tables, used in ALTER SESSION
SET CURRENT_SCHEMA
     LOOKBACK_DAYS    runtime window in days for queries 03c, 09, 09b, 10 (form
asks for 30/60/90)
     TARGET_SCHEMAS   quoted, comma-separated DWH schema list for query 11, e.g.
'ODS','DWH_STG','DWH','DM'
                       (physical schema names, query 07c shows which physical
schema each ODI
                       model points to)

   VERSION WARNING
     Repository column names differ between ODI 11g and 12c and between 12c
patch levels.
     The table names used here are the 12c ones (SNP_MAPPING, SNP_MAP_COMP,
SNP_TXTHEADER...).
     11g repositories use SNP_POP / SNP_SOURCE_TAB for interfaces and SNP_TXT
(250-character
     chunks) for text, those are not covered. Every column that is not certain
is marked
     "-- VERIFY:" with the most likely alternative. Query 00 lists the columns
this script
     expects and whether they exist, so a mismatch is visible before anything
else runs.
     Runtime queries only see what the Operator log still holds: if logs are
purged,
     the window is shorter than LOOKBACK_DAYS (09 prints the first run date per
scenario).

================================================================================
===== */

SET DEFINE ON
SET VERIFY OFF
SET FEEDBACK OFF
SET PAGESIZE 50000
SET LINESIZE 32767
SET TRIMSPOOL ON
SET LONG 1000000
SET LONGCHUNKSIZE 1000000
SET SQLBLANKLINES ON
SET SQLFORMAT CSV
WHENEVER SQLERROR CONTINUE

DEFINE ODI_WORK_SCHEMA = DEV_ODI_REPO
DEFINE LOOKBACK_DAYS   = 90
DEFINE TARGET_SCHEMAS  = "'ODS','DWH_STG','DWH','DM'"

ALTER SESSION SET CURRENT_SCHEMA = &ODI_WORK_SCHEMA;

--
================================================================================
=====
-- 00  SCHEMA CHECK                                        (no form sheet,
prerequisite)
--
================================================================================
=====

-- 00a  SNP_ tables present in the repository schema, with optimizer row counts.
--      Large SNP_SESS_TASK_LOG / SNP_SESSION counts mean long log retention
```

```
(good for 09/10).
SPOOL odi_q00a_snp_tables.csv
SELECT t.table_name,
       t.num_rows,
       t.last_analyzed
FROM   all_tables t
WHERE  t.owner = SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA')
AND    t.table_name LIKE 'SNP\_%' ESCAPE '\'
ORDER  BY t.table_name;
SPOOL OFF

-- 00b  Every table.column this script uses, and whether it exists.
--      STATUS = COLUMN MISSING -> read AVAILABLE_COLUMNS, then fix the query
lines marked VERIFY.
--      DATA_TYPE tells whether ERROR_MESSAGE / FULL_TEXT are CLOB or VARCHAR2.
SPOOL odi_q00b_column_check.csv
WITH expected AS (
    SELECT REGEXP_SUBSTR(COLUMN_VALUE, '[^.|]+', 1, 1) AS table_name,
           REGEXP_SUBSTR(COLUMN_VALUE, '[^.|]+', 1, 2) AS column_name,
           REGEXP_SUBSTR(COLUMN_VALUE, '[^|]+',  1, 2) AS used_by_queries
    FROM   TABLE(sys.odcivarchar2list(
        'SNP_PROJECT.I_PROJECT|01 02 03 04 05 07 12',
        'SNP_PROJECT.PROJECT_NAME|01 02 03 04 05 12',
        'SNP_FOLDER.I_FOLDER|01 02 03 04 07 12',
        'SNP_FOLDER.I_PROJECT|01 02 03 04 07 12',
        'SNP_FOLDER.PAR_I_FOLDER|01 02 03 04 07 12',
        'SNP_FOLDER.FOLDER_NAME|01 02 03 04 07 12',
        'SNP_PACKAGE.I_PACKAGE|all',
        'SNP_PACKAGE.I_FOLDER|01 02 07 12',
        'SNP_PACKAGE.PACK_NAME|all',
        'SNP_PACKAGE.LAST_DATE|01',
        'SNP_PACKAGE.LAST_USER|01',
        'SNP_STEP.I_PACKAGE|01 02 04 05 06 12',
        'SNP_STEP.NNO|06',
        'SNP_STEP.STEP_NAME|06',
        'SNP_STEP.STEP_TYPE|01 02 05 06 12',
        'SNP_STEP.I_MAPPING|01 02 03 07 12',
        'SNP_STEP.I_TRT|01 02 04 12',
        'SNP_STEP.I_VAR|01 02 05 12',
        'SNP_MAPPING.I_MAPPING|01 03 12',
        'SNP_MAPPING.NAME|03',
        'SNP_MAPPING.I_FOLDER|03',
        'SNP_MAPPING.LAST_DATE|01 03',
        'SNP_MAP_COMP.I_MAP_COMP|01 02 03 07 12',
        'SNP_MAP_COMP.I_OWNER_MAPPING|01 02 03 07 12',
        'SNP_MAP_COMP.NAME|01 03',
        'SNP_MAP_COMP.I_MAP_REF|01 02 07 12',
        'SNP_MAP_COMP.I_MAP_COMP_TYPE|01 02 03 07 12',
        'SNP_MAP_COMP_TYPE.I_MAP_COMP_TYPE|01 02 03 07 12',
        'SNP_MAP_COMP_TYPE.TYPE_NAME|01 02 03 07 12',
        'SNP_MAP_CP.I_MAP_CP|01 02 07 12',
        'SNP_MAP_CP.I_OWNER_MAP_COMP|01 02 07 12',
        'SNP_MAP_CONN.I_END_MAP_CP|01 02 07 12',
        'SNP_MAP_REF.I_MAP_REF|01 02 07 12',
        'SNP_MAP_REF.I_REF_ID|01 02 07 12',
        'SNP_MAP_REF.QUALIFIED_NAME|01 02 07 12',
        'SNP_MAP_REF.ADAPTER_INTF_TYPE|00d',
        'SNP_TABLE.I_TABLE|01 02 06 07 12',
        'SNP_TABLE.I_MOD|01 02 06 07 12',
        'SNP_TABLE.TABLE_NAME|01 02 06 07 12',
        'SNP_TABLE.IND_JRN|06',
        'SNP_MODEL.I_MOD|01 02 06 07 12',
        'SNP_MODEL.COD_MOD|01 02 06 07 12',
        'SNP_MODEL.MOD_NAME|06 07',
```

```
'SNP_MODEL.TECH_INT_NAME|01 02 06 07 12',
'SNP_MODEL.LSCHEMA_NAME|01 02 06 07 12',
'SNP_MODEL.I_TRT_KJM|06',
'SNP_TRT.I_TRT|04 06',
'SNP_TRT.TRT_NAME|04 06',
'SNP_TRT.TRT_TYPE|04',
'SNP_TRT.I_FOLDER|04',
'SNP_TRT.I_PROJECT|04 06',
'SNP_TRT.LAST_DATE|04 06',
'SNP_LINE_TRT.I_TRT|04 05',
'SNP_LINE_TRT.ORD_TRT|04',
'SNP_LINE_TRT.SQL_NAME|04',
'SNP_LINE_TRT.DEF_I_TXT|04 05',
'SNP_LINE_TRT.COL_I_TXT|04',
'SNP_LINE_TRT.DEF_TECH_INT_NAME|04',
'SNP_LINE_TRT.COL_TECH_INT_NAME|04',
'SNP_LINE_TRT.DEF_LSCHEMA_NAME|04',
'SNP_LINE_TRT.COL_LSCHEMA_NAME|04',
'SNP_LINE_TRT.IND_ERR|04',
'SNP_TXTHEADER.I_TXT|04 05',
'SNP_TXTHEADER.FULL_TEXT|04 05',
'SNP_VAR.I_VAR|05',
'SNP_VAR.VAR_NAME|05',
'SNP_VAR.I_PROJECT|05',
'SNP_VAR.VAR_DATATYPE|05',
'SNP_VAR.IND_STORE|05',
'SNP_VAR.DEF_V|05',
'SNP_VAR.LSCHEMA_NAME|05',
'SNP_VAR.I_TXT_VAR_IN|05',
'SNP_VAR.LAST_DATE|05',
'SNP_SCEN.SCEN_NO|03 07',
'SNP_SCEN.SCEN_NAME|01 03 07 08 09 10',
'SNP_SCEN.SCEN_VERSION|01',
'SNP_SCEN.I_PACKAGE|01 03 07 08 09 10',
'SNP_SCEN_STEP.SCEN_NO|03',
'SNP_SCEN_STEP.NNO|03',
'SNP_SCEN_STEP.STEP_NAME|03',
'SNP_SCEN_TASK.SCEN_NO|03 06 07',
'SNP_SCEN_TASK.NNO|03 07',
'SNP_SCEN_TASK.TASK_NAME1|03 06 07',
'SNP_SCEN_TASK.TASK_NAME2|03 06 07',
'SNP_SCEN_TASK.TASK_NAME3|03 06 07',
'SNP_SCEN_TASK.DEF_TECH_INT_NAME|07',
'SNP_SCEN_TASK.COL_TECH_INT_NAME|07',
'SNP_LOAD_PLAN.I_LOAD_PLAN|01 08',
'SNP_LOAD_PLAN.LOAD_PLAN_NAME|01 08',
'SNP_LP_STEP.I_LP_STEP|08',
'SNP_LP_STEP.I_LOAD_PLAN|01 08',
'SNP_LP_STEP.PAR_I_LP_STEP|08',
'SNP_LP_STEP.LP_STEP_NAME|08',
'SNP_LP_STEP.LP_STEP_TYPE|08',
'SNP_LP_STEP.STEP_ORDER|08',
'SNP_LP_STEP.IND_ENABLED|01 08',
'SNP_LP_STEP.SCEN_NAME|01 08',
'SNP_LP_STEP.SCEN_VERSION|08',
'SNP_LP_STEP.RESTART_TYPE|08',
'SNP_PLAN_AGENT.SCEN_NAME|01 08',
'SNP_PLAN_AGENT.SCEN_VERSION|08',
'SNP_PLAN_AGENT.I_LOAD_PLAN|01 08',
'SNP_PLAN_AGENT.AGENT_NAME|01 08',
'SNP_PLAN_AGENT.STAT_PLAN|01 08',
'SNP_PLAN_AGENT.S_TYPE|01 08',
'SNP_PLAN_AGENT.S_HOUR|01 08',
'SNP_PLAN_AGENT.S_MINUTE|01 08',
```

```
        'SNP_SESSION.SESS_NO|03 09 10',
        'SNP_SESSION.SESS_NAME|09 10',
        'SNP_SESSION.SCEN_NAME|09 10',
        'SNP_SESSION.SESS_BEG|03 09 10',
        'SNP_SESSION.SESS_END|09',
        'SNP_SESSION.SESS_DUR|09',
        'SNP_SESSION.SESS_STATUS|09',
        'SNP_SESSION.ERROR_MESSAGE|09',
        'SNP_SESS_STEP.SESS_NO|10',
        'SNP_SESS_STEP.NNO|10',
        'SNP_SESS_STEP.STEP_NAME|10',
        'SNP_SESS_TASK.SESS_NO|03',
        'SNP_SESS_TASK.TASK_NAME1|03',
        'SNP_SESS_TASK.TASK_NAME2|03',
        'SNP_SESS_TASK.TASK_NAME3|03',
        'SNP_SESS_TASK_LOG.SESS_NO|10',
        'SNP_SESS_TASK_LOG.NNO|10',
        'SNP_SESS_TASK_LOG.NB_ROW|10',
        'SNP_SESS_TASK_LOG.NB_INS|10',
        'SNP_SESS_TASK_LOG.NB_UPD|10',
        'SNP_SESS_TASK_LOG.NB_DEL|10',
        'SNP_SESS_TASK_LOG.NB_ERR|10',
        'SNP_STEP_LOG.SESS_NO|10',
        'SNP_STEP_LOG.NNO|10',
        'SNP_STEP_LOG.NB_INS|10',
        'SNP_STEP_LOG.NB_UPD|10',
        'SNP_STEP_LOG.NB_DEL|10',
        'SNP_STEP_LOG.NB_ROW|10',
        'SNP_LP_INST.I_LP_INST|09',
        'SNP_LP_INST.LOAD_PLAN_NAME|09',
        'SNP_LPI_RUN.I_LP_INST|09',
        'SNP_LPI_RUN.NB_RUN|09',
        'SNP_LPI_RUN.START_DATE|09',
        'SNP_LPI_RUN.END_DATE|09',
        'SNP_LPI_RUN.STATUS|09',
        'SNP_LPI_RUN.ERROR_MESSAGE|09',
        'SNP_LSCHEMA.I_LSCHEMA|07',
        'SNP_LSCHEMA.LSCHEMA_NAME|07',
        'SNP_PSCHEMA_CONT.I_LSCHEMA|07',
        'SNP_PSCHEMA_CONT.I_PSCHEMA|07',
        'SNP_PSCHEMA_CONT.I_CONTEXT|07',
        'SNP_PSCHEMA.I_PSCHEMA|07',
        'SNP_PSCHEMA.I_CONNECT|07',
        'SNP_PSCHEMA.SCHEMA_NAME|07',
        'SNP_CONNECT.I_CONNECT|07',
        'SNP_CONNECT.CON_NAME|07',
        'SNP_CONNECT.I_TECHNO|07',
        'SNP_TECHNO.I_TECHNO|07',
        'SNP_TECHNO.TECH_INT_NAME|07',
        'SNP_CONTEXT.I_CONTEXT|07',
        'SNP_CONTEXT.CONTEXT_CODE|07'
    ))
)
SELECT e.table_name,
       e.column_name,
       e.used_by_queries,
       CASE
         WHEN NOT EXISTS (SELECT 1 FROM all_tab_columns x
                          WHERE  x.owner = SYS_CONTEXT('USERENV',
'CURRENT_SCHEMA')
                          AND    x.table_name = e.table_name) THEN 'TABLE
MISSING'
         WHEN c.column_name IS NULL                                THEN 'COLUMN
MISSING'
```

```
         ELSE 'OK'
       END AS status,
       c.data_type,
       CASE WHEN c.column_name IS NULL THEN
            (SELECT LISTAGG(x.column_name, ', ' ON OVERFLOW TRUNCATE '...' WITH
COUNT)
                    WITHIN GROUP (ORDER BY x.column_id)
             FROM   all_tab_columns x
             WHERE  x.owner = SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA')
             AND    x.table_name = e.table_name)
       END AS available_columns
FROM   expected e
LEFT JOIN all_tab_columns c
       ON c.owner       = SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA')
      AND c.table_name  = e.table_name
      AND c.column_name = e.column_name
ORDER  BY CASE WHEN c.column_name IS NULL THEN 0 ELSE 1 END, e.table_name,
e.column_name;
SPOOL OFF

-- 00c  Candidate columns for the uncertain joins: text pointers (I_TXT*),
technology,
--      KM references (I_TRT*), mapping physical design tables. Use it to fix
VERIFY lines.
SPOOL odi_q00c_candidate_columns.csv
SELECT c.table_name,
       c.column_name,
       c.data_type
FROM   all_tab_columns c
WHERE  c.owner = SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA')
AND    c.table_name LIKE 'SNP\_%' ESCAPE '\'
AND   (   c.column_name LIKE 'I\_TXT%' ESCAPE '\'
       OR c.column_name LIKE '%\_I\_TXT' ESCAPE '\'
       OR c.column_name LIKE '%TECH%'
       OR c.column_name LIKE 'I\_TRT%' ESCAPE '\'
       OR c.column_name LIKE '%KM%'
       OR c.column_name LIKE '%JRN%'
       OR c.table_name  LIKE 'SNP\_MAP%' ESCAPE '\'
       OR c.table_name  LIKE 'SNP\_PHY%' ESCAPE '\'
       OR c.table_name  LIKE 'SNP\_DEP%' ESCAPE '\'
       OR c.table_name  LIKE 'SNP\_EXEC\_UNIT%' ESCAPE '\'
       OR c.table_name  IN ('SNP_PLAN_AGENT', 'SNP_LP_STEP', 'SNP_VAR',
'SNP_LINE_TRT'))
ORDER  BY c.table_name, c.column_id;
SPOOL OFF

-- 00d  Value checks. Each is independent, if one fails, the column it reads is
wrong.
-- 00d-1 Mapping component type names (drives the comp_class CASE in
01/02/03/12).
SPOOL odi_q00d1_component_types.csv
SELECT ct.type_name,                      -- VERIFY: SNP_MAP_COMP_TYPE.TYPE_NAME
(alt: INTERNAL_NAME, NAME)
       COUNT(c.i_map_comp) AS component_cnt
FROM   snp_map_comp_type ct
LEFT JOIN snp_map_comp c ON c.i_map_comp_type = ct.i_map_comp_type
GROUP  BY ct.type_name
ORDER  BY component_cnt DESC;
SPOOL OFF

-- 00d-2 Package step types with counts and example names (drives
tool/journalizing step codes).
SPOOL odi_q00d2_step_types.csv
SELECT s.step_type,
```

```
       COUNT(*)                                                AS step_cnt,
       COUNT(s.i_mapping)                                      AS with_mapping,
       COUNT(s.i_trt)                                          AS with_procedure,
       COUNT(s.i_var)                                          AS with_variable,
       MIN(s.step_name)                                        AS example_step_1,
       MAX(s.step_name)                                        AS example_step_2
FROM   snp_step s
GROUP  BY s.step_type
ORDER  BY step_cnt DESC;
SPOOL OFF

-- 00d-3 What SNP_MAP_REF points to (datastore refs should resolve to
SNP_TABLE).
SPOOL odi_q00d3_map_ref_types.csv
SELECT r.adapter_intf_type,              -- VERIFY: column name
ADAPTER_INTF_TYPE (alt: REF_TYPE)
       COUNT(*)                                                AS ref_cnt,
       COUNT(t.i_table)                                        AS
resolves_to_snp_table,
       MIN(r.qualified_name)                                   AS
example_qualified_name
FROM   snp_map_ref r
LEFT JOIN snp_table t ON t.i_table = r.i_ref_id
GROUP  BY r.adapter_intf_type
ORDER  BY ref_cnt DESC;
SPOOL OFF

-- 00d-4 Load plan step type codes and session status codes.
SPOOL odi_q00d4_codes.csv
SELECT 'SNP_LP_STEP.LP_STEP_TYPE' AS code_column, TO_CHAR(lp_step_type) AS code,
COUNT(*) AS cnt
FROM   snp_lp_step GROUP BY lp_step_type
UNION ALL
SELECT 'SNP_SESSION.SESS_STATUS', TO_CHAR(sess_status), COUNT(*)
FROM   snp_session GROUP BY sess_status
UNION ALL
SELECT 'SNP_LP_STEP.IND_ENABLED', TO_CHAR(ind_enabled), COUNT(*)
FROM   snp_lp_step GROUP BY ind_enabled
ORDER  BY 1, 3 DESC;
SPOOL OFF

--
================================================================================
=====
-- 01  PACKAGE INVENTORY                        -> sheet "3 - Pipeline & Job
Envanteri"
--     # = ROW_NO, Job Adi = JOB_NAME, ODI Tipi = ODI_TYPE, Kaynak =
SOURCE_DATASTORES,
--     Hedef = TARGET_DATASTORES, Schedule = LOAD_PLANS / DIRECT_SCHEDULES.
--     Islev, Extract/Transform/Load, Ort. Sure, Kritik Mi? stay manual (02
suggests E/T/L,
--     09 gives the average duration).
--     Sources include intermediate tables written by another mapping of the
same package,
--     EXT_SOURCE_CNT excludes them.
--
================================================================================
=====
SPOOL odi_q01_package_inventory.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
```

```
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
pkg AS (
    SELECT p.i_package,
           p.pack_name,
           pr.project_name,
           fld.folder_path,
           p.last_date,
           p.last_user
    FROM   snp_package p
    LEFT JOIN fld         ON fld.i_folder  = p.i_folder
    LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
             WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
             WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
             WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
             WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
             WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
             WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
             WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
             WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
             WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
             WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
             WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
             WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
             WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
             WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
             WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
             WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
             ELSE 'OTHER'
           END                         AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
SOURCE
```

```
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                    SELECT 1
                    FROM   snp_map_cp   cp
                    JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                    WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
),
pagg AS (
    SELECT i_package,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' THEN ds_name END)
AS src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND also_target = 0 THEN
ds_name END)  AS ext_src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'TARGET' THEN ds_name END)
AS tgt_cnt,
           -- Assumes the target DWH is Oracle (MACFit). Change the literal for
other estates.
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND NVL(technology, '?')
<> 'ORACLE'
                               THEN ds_name END)
AS src_non_oracle_cnt
    FROM   pds
    GROUP  BY i_package
```

```
),
pstep AS (
    -- Step mix per package. Classified by which object column is filled, which
is more
    -- stable across versions than STEP_TYPE codes.
    SELECT s.i_package,
           COUNT(*)
AS step_cnt,
           COUNT(DISTINCT s.i_mapping)
AS mapping_cnt,
           COUNT(CASE WHEN s.i_trt IS NOT NULL THEN 1 END)
AS procedure_step_cnt,
           COUNT(DISTINCT s.i_trt)
AS procedure_cnt,
           COUNT(CASE WHEN s.i_var IS NOT NULL THEN 1 END)
AS variable_step_cnt,
           COUNT(DISTINCT s.i_var)
AS variable_cnt,
           COUNT(CASE WHEN s.step_type IN ('SE', 'OE') THEN 1 END)
AS tool_step_cnt,   -- VERIFY: STEP_TYPE 'SE' = ODI tool (OdiStartScen,
OdiFileMove...), 'OE' = OS command
           COUNT(CASE WHEN s.i_mapping IS NULL AND s.i_trt IS NULL AND s.i_var
IS NULL
                      THEN 1 END)
AS other_step_cnt
    FROM   snp_step s
    GROUP  BY s.i_package
)
,
plist AS (
    SELECT i_package,
           LISTAGG(CASE WHEN ds_role = 'SOURCE' THEN ds_name END, ', ' ON
OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY ds_name) AS src_list,
           LISTAGG(CASE WHEN ds_role = 'TARGET' THEN ds_name END, ', ' ON
OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY ds_name) AS tgt_list
    FROM  (SELECT DISTINCT i_package, ds_role, ds_name FROM pds)
    GROUP  BY i_package
),
ptech AS (
    SELECT i_package,
           LISTAGG(technology, ', ') WITHIN GROUP (ORDER BY technology) AS
src_tech_list
    FROM  (SELECT DISTINCT i_package, technology FROM pds WHERE ds_role =
'SOURCE' AND technology IS NOT NULL)
    GROUP  BY i_package
),
pmlast AS (
    SELECT pm.i_package, MAX(m.last_date) AS mapping_last_date
    FROM   pmap pm
    JOIN   snp_mapping m ON m.i_mapping = pm.i_mapping
    GROUP  BY pm.i_package
),
scn AS (
    -- Scenarios generated from each package (scenario name is what load plans
and schedules use).
    SELECT sc.i_package, sc.scen_name, MAX(sc.scen_version) AS last_scen_version
    FROM   snp_scen sc
    WHERE  sc.i_package IS NOT NULL
    GROUP  BY sc.i_package, sc.scen_name
),
sched AS (
    -- One text per schedule row. SNP_PLAN_AGENT varies most between versions,
```

```
so it is isolated
    -- here. If this CTE fails, replace its body with:
    --   SELECT CAST(NULL AS VARCHAR2(400)) scen_name, CAST(NULL AS
VARCHAR2(400)) load_plan_name,
    --          CAST(NULL AS VARCHAR2(400)) sched_txt FROM dual WHERE 1 = 0
    -- and take schedules from 08b (SELECT * FROM SNP_PLAN_AGENT) instead.
    SELECT pa.scen_name,
           lp.load_plan_name,
           'status=' || pa.stat_plan                                        --
VERIFY: STAT_PLAN codes (active/inactive) from 08b
           || ' type=' || pa.s_type                                         --
VERIFY: S_TYPE (alt: EXEC_TYPE / PLAN_TYPE) = daily/hourly/...
           || ' at ' || LPAD(TO_CHAR(pa.s_hour), 2, '0')                    --
VERIFY: S_HOUR / S_MINUTE (alt: START_HOUR / START_MINUTE)
           || ':'    || LPAD(TO_CHAR(pa.s_minute), 2, '0')
           || ' agent=' || pa.agent_name                                     AS
sched_txt
    FROM   snp_plan_agent pa
    LEFT JOIN snp_load_plan lp ON lp.i_load_plan = pa.i_load_plan          --
VERIFY: SNP_PLAN_AGENT.I_LOAD_PLAN (alt: load plan name in SCEN_NAME with a type
flag)
),
lpuse AS (
    -- Load plans that run each scenario, with the load plan schedule.
    SELECT x.scen_name,
           LISTAGG(x.lp_txt, ' | ' ON OVERFLOW TRUNCATE '...' WITH COUNT) WITHIN
GROUP (ORDER BY x.lp_txt) AS lp_list
    FROM  (SELECT DISTINCT
                  ls.scen_name,                                              --
VERIFY: SNP_LP_STEP.SCEN_NAME
                  lp.load_plan_name
                  || CASE WHEN TO_CHAR(ls.ind_enabled) IN ('0', 'N') THEN '
[step disabled]' END   -- VERIFY: IND_ENABLED values (00d-4)
                  || NVL2(sc.sched_txt, ' @ ' || sc.sched_txt, ' [no LP
schedule]') AS lp_txt
           FROM   snp_lp_step   ls
           JOIN   snp_load_plan lp ON lp.i_load_plan = ls.i_load_plan
           LEFT JOIN sched      sc ON sc.load_plan_name = lp.load_plan_name
           WHERE  ls.scen_name IS NOT NULL) x
    GROUP  BY x.scen_name
),
dsched AS (
    -- Schedules attached directly to the scenario (not through a load plan).
    SELECT scen_name,
           LISTAGG(sched_txt, ' | ' ON OVERFLOW TRUNCATE '...' WITH COUNT)
WITHIN GROUP (ORDER BY sched_txt) AS sched_list
    FROM  (SELECT DISTINCT scen_name, sched_txt FROM sched WHERE scen_name IS
NOT NULL AND load_plan_name IS NULL)
    GROUP  BY scen_name
),
pscn AS (
    SELECT scn.i_package,
           LISTAGG(scn.scen_name || ' v' || scn.last_scen_version, ', ' ON
OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY scn.scen_name) AS scen_list,
           LISTAGG(lu.lp_list, ' | ' ON OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY scn.scen_name) AS lp_list,
           LISTAGG(dsc.sched_list, ' | ' ON OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY scn.scen_name) AS direct_sched_list
    FROM   scn
    LEFT JOIN lpuse  lu  ON lu.scen_name  = scn.scen_name
    LEFT JOIN dsched dsc ON dsc.scen_name = scn.scen_name
    GROUP  BY scn.i_package
)
```

```
SELECT ROW_NUMBER() OVER (ORDER BY pkg.pack_name)          AS row_no,
-- sheet 3: #
       pkg.pack_name                                       AS job_name,
-- sheet 3: Job Adi
       'Package'                                           AS odi_type,
-- sheet 3: ODI Tipi
       pkg.project_name,
       pkg.folder_path,
       NVL(ps.step_cnt, 0)                                 AS step_cnt,
       NVL(ps.mapping_cnt, 0)                              AS mapping_cnt,
       NVL(ps.procedure_cnt, 0)                            AS procedure_cnt,
       NVL(ps.procedure_step_cnt, 0)                       AS
procedure_step_cnt,
       NVL(ps.variable_cnt, 0)                             AS variable_cnt,
       NVL(ps.variable_step_cnt, 0)                        AS variable_step_cnt,
       NVL(ps.tool_step_cnt, 0)                            AS tool_step_cnt,
       pl.src_list                                         AS source_datastores,
-- sheet 3: Kaynak
       CASE WHEN NVL(ps.mapping_cnt, 0) = 0
            THEN 'No mapping step (procedure / variable / other steps)'
            ELSE pl.tgt_list
       END                                                 AS target_datastores,
-- sheet 3: Hedef
       NVL(pa.src_cnt, 0)                                  AS source_cnt,
       NVL(pa.ext_src_cnt, 0)                              AS ext_source_cnt,
       NVL(pa.tgt_cnt, 0)                                  AS target_cnt,
       pt.src_tech_list                                    AS
source_technologies,
       pc.scen_list                                        AS scenarios,
       pc.lp_list                                          AS load_plans,
-- sheet 3: Schedule (via load plan)
       pc.direct_sched_list                                AS direct_schedules,
-- sheet 3: Schedule (scenario scheduled directly)
       CASE WHEN pc.lp_list IS NULL AND pc.direct_sched_list IS NULL
            THEN 'Not in a load plan or schedule - check (called by
OdiStartScen? inactive?)'
       END                                                 AS schedule_check,
       CASE WHEN REGEXP_LIKE(pkg.pack_name, 'TEST|_TST|DENEME|_OLD|_BCK|_YEDEK',
'i')
            THEN 'Y' END                                   AS test_or_old_name,
       pkg.last_date                                       AS
package_last_modified,
       pkg.last_user                                       AS package_last_user,
       pm.mapping_last_date                                AS
mappings_last_modified
FROM   pkg
LEFT JOIN pstep  ps ON ps.i_package = pkg.i_package
LEFT JOIN pagg   pa ON pa.i_package = pkg.i_package
LEFT JOIN plist  pl ON pl.i_package = pkg.i_package
LEFT JOIN ptech  pt ON pt.i_package = pkg.i_package
LEFT JOIN pmlast pm ON pm.i_package = pkg.i_package
LEFT JOIN pscn   pc ON pc.i_package = pkg.i_package
ORDER  BY pkg.pack_name;
SPOOL OFF

--
================================================================================
=====
-- 02  EFFORT GROUP CLASSIFICATION (Rev2 rule)  -> plan.xlsx "ODI Dagilimi"
counts,
--     also suggests sheet 3 Extract / Transform / Load columns.
--     BLIND_SPOT_PROFILE breaks down groups 1-2 (the 34% blind spot) by what
the package
--     actually contains. DATASTREAM_CANDIDATE marks 1:1 copy jobs that should
```

```
become
--     Datastream / generic landing configuration instead of being converted one
by one.
--
================================================================================
=====
SPOOL odi_q02_effort_groups.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
pkg AS (
    SELECT p.i_package,
           p.pack_name,
           pr.project_name,
           fld.folder_path,
           p.last_date,
           p.last_user
    FROM   snp_package p
    LEFT JOIN fld         ON fld.i_folder  = p.i_folder
    LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
             WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
             WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
             WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
             WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
             WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
             WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
             WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
             WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
             WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
             WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
             WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
             WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
             WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
             WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
             WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
             WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
```

```
            ELSE 'OTHER'
          END                          AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
SOURCE
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                    SELECT 1
                    FROM   snp_map_cp   cp
                    JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                    WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
),
pagg AS (
    SELECT i_package,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' THEN ds_name END)
AS src_cnt,
```

```
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND also_target = 0 THEN
ds_name END)  AS ext_src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'TARGET' THEN ds_name END)
AS tgt_cnt,
           -- Assumes the target DWH is Oracle (MACFit). Change the literal for
other estates.
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND NVL(technology, '?')
<> 'ORACLE'
                                THEN ds_name END)
AS src_non_oracle_cnt
    FROM   pds
    GROUP  BY i_package
),
pstep AS (
    -- Step mix per package. Classified by which object column is filled, which
is more
    -- stable across versions than STEP_TYPE codes.
    SELECT s.i_package,
           COUNT(*)
AS step_cnt,
           COUNT(DISTINCT s.i_mapping)
AS mapping_cnt,
           COUNT(CASE WHEN s.i_trt IS NOT NULL THEN 1 END)
AS procedure_step_cnt,
           COUNT(DISTINCT s.i_trt)
AS procedure_cnt,
           COUNT(CASE WHEN s.i_var IS NOT NULL THEN 1 END)
AS variable_step_cnt,
           COUNT(DISTINCT s.i_var)
AS variable_cnt,
           COUNT(CASE WHEN s.step_type IN ('SE', 'OE') THEN 1 END)
AS tool_step_cnt,   -- VERIFY: STEP_TYPE 'SE' = ODI tool (OdiStartScen,
OdiFileMove...), 'OE' = OS command
           COUNT(CASE WHEN s.i_mapping IS NULL AND s.i_trt IS NULL AND s.i_var
IS NULL
                      THEN 1 END)
AS other_step_cnt
    FROM   snp_step s
    GROUP  BY s.i_package
),
pcomp AS (
    SELECT pm.i_package,
           COUNT(CASE WHEN comp.comp_class IN ('JOIN', 'LOOKUP', 'AGGREGATE',
'SET', 'PIVOT', 'UNPIVOT',
                                               'SPLIT', 'SUBQUERY_FILTER',
'REUSABLE_MAPPING', 'TABLE_FUNCTION')
                      THEN 1 END)
AS heavy_comp_cnt,
           COUNT(CASE WHEN comp.comp_class NOT IN ('DATASTORE', 'SIGNATURE',
'DATASET') THEN 1 END) AS transform_comp_cnt
    FROM   pmap pm
    JOIN   comp ON comp.i_mapping = pm.i_mapping
    GROUP  BY pm.i_package
),
cls AS (
    -- Rev2 effort-group rule, evaluated in this order:
    --   1 mapping not visible   : package has no mapping step
    --   2 target unclear        : mapping steps exist but no target datastore
resolved
    --   3 5+ sources or multiple targets
    --   4 2-4 sources, single target
    --   5 single source, single target (also catches 0 sources + 1 target, e.g.
truncate/insert-only)
    SELECT pkg.i_package,
```

```
           pkg.pack_name,
           pkg.project_name,
           pkg.folder_path,
           REGEXP_SUBSTR(pkg.pack_name, '^[^_]+_[^_]+')           AS
name_prefix,   -- e.g. PKG_EXT, PCK_ODS
           NVL(ps.step_cnt, 0)            AS step_cnt,
           NVL(ps.mapping_cnt, 0)         AS mapping_cnt,
           NVL(ps.procedure_step_cnt, 0)  AS procedure_step_cnt,
           NVL(ps.procedure_cnt, 0)       AS procedure_cnt,
           NVL(ps.variable_step_cnt, 0)   AS variable_step_cnt,
           NVL(ps.tool_step_cnt, 0)       AS tool_step_cnt,
           NVL(ps.other_step_cnt, 0)      AS other_step_cnt,
           NVL(pa.src_cnt, 0)             AS src_cnt,
           NVL(pa.ext_src_cnt, 0)         AS ext_src_cnt,
           NVL(pa.tgt_cnt, 0)             AS tgt_cnt,
           NVL(pa.src_non_oracle_cnt, 0)  AS src_non_oracle_cnt,
           NVL(pc.heavy_comp_cnt, 0)      AS heavy_comp_cnt,
           NVL(pc.transform_comp_cnt, 0)  AS transform_comp_cnt,
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN '1
Mapping not visible'
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN '2
Target unclear'
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN '3 5+
sources or multiple targets'
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN '4 2-4
sources / single target'
             ELSE                                                          '5
Single source / single target'
           END                            AS effort_group,
           -- Rev2 unit hours (analysis + conversion + test). Assumptions,
calibrate after the pilot.
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN 5.5
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN 5.5
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN 8
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN 5.5
             ELSE                                                          3
           END                            AS rev2_unit_hours
    FROM   pkg
    LEFT JOIN pstep ps ON ps.i_package = pkg.i_package
    LEFT JOIN pagg  pa ON pa.i_package = pkg.i_package
    LEFT JOIN pcomp pc ON pc.i_package = pkg.i_package
)
SELECT cls.pack_name,
       cls.name_prefix,
       cls.project_name,
       cls.folder_path,
       cls.effort_group,
       cls.rev2_unit_hours,
       cls.step_cnt,
       cls.mapping_cnt,
       cls.procedure_cnt,
       cls.procedure_step_cnt,
       cls.variable_step_cnt,
       cls.tool_step_cnt,
       cls.other_step_cnt,
       cls.src_cnt,
       cls.ext_src_cnt,
       cls.tgt_cnt,
       cls.src_non_oracle_cnt,
       cls.heavy_comp_cnt,
       cls.transform_comp_cnt,
       CASE
```

```
         WHEN cls.effort_group LIKE '1%' THEN
           CASE
             WHEN cls.step_cnt = 0
THEN 'Empty package'
             WHEN cls.procedure_step_cnt > 0 AND cls.variable_step_cnt = 0
                  AND cls.other_step_cnt = 0
THEN 'Procedure only'
             WHEN cls.procedure_step_cnt > 0
THEN 'Procedure + variables / tools'
             WHEN cls.tool_step_cnt > 0
THEN 'Tool steps only (OdiStartScen / OS / file tools) - orchestrator?'
             WHEN cls.variable_step_cnt > 0 AND cls.other_step_cnt = 0
THEN 'Variables only (loop / control)'
             ELSE
'Other step types (model / journalizing / check)'
           END
         WHEN cls.effort_group LIKE '2%' THEN
           'Mappings exist but no target resolved (reusable mapping / unresolved
datastore ref) - check 03'
       END                                                              AS
blind_spot_profile,
       CASE
         WHEN cls.effort_group LIKE '5%' AND cls.heavy_comp_cnt = 0
              AND (cls.src_non_oracle_cnt > 0 OR cls.name_prefix IN ('PKG_EXT',
'PCK_EXT', 'PKG_ODS', 'PCK_ODS'))
         THEN 'Y'
       END                                                              AS
datastream_candidate,
       -- Suggestions for sheet 3, the vendor/customer confirms them.
       CASE WHEN cls.src_non_oracle_cnt > 0 THEN 'Evet' ELSE 'Hayir' END
AS suggested_extract,
       CASE WHEN cls.transform_comp_cnt > 0 OR cls.procedure_step_cnt > 0
            THEN 'Evet' ELSE 'Hayir' END
AS suggested_transform,
       CASE WHEN cls.tgt_cnt > 0 OR cls.procedure_step_cnt > 0 THEN 'Evet' ELSE
'Hayir' END AS suggested_load
FROM   cls
ORDER  BY cls.effort_group, cls.pack_name;
SPOOL OFF

--
================================================================================
=====
-- 03  MAPPING COMPLEXITY AND KNOWLEDGE MODULES  (no form sheet, feeds unit-hour
calibration)
--
================================================================================
=====

-- 03a  Component counts per mapping. COMPLEXITY_BAND is a first-cut heuristic
on the
--      number of transform components, calibrate it against the pilot.
SPOOL odi_q03a_mapping_components.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
```

```
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping          AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                     AS comp_name,
           c.i_map_ref,                               -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)        AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
             WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
             WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
             WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
             WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
             WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
             WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
             WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
             WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
             WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
             WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
             WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
             WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
             WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
             WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
             WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
             WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
             ELSE 'OTHER'
           END                        AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
)
,
mcnt AS (
    SELECT comp.i_mapping,
           COUNT(CASE WHEN comp_class = 'DATASTORE'        THEN 1 END) AS
datastore_cnt,
           COUNT(CASE WHEN comp_class = 'JOIN'             THEN 1 END) AS
join_cnt,
           COUNT(CASE WHEN comp_class = 'FILTER'           THEN 1 END) AS
filter_cnt,
           COUNT(CASE WHEN comp_class = 'EXPRESSION'       THEN 1 END) AS
expression_cnt,
           COUNT(CASE WHEN comp_class = 'AGGREGATE'        THEN 1 END) AS
aggregate_cnt,
           COUNT(CASE WHEN comp_class = 'LOOKUP'           THEN 1 END) AS
lookup_cnt,
           COUNT(CASE WHEN comp_class = 'SET'              THEN 1 END) AS
set_cnt,
           COUNT(CASE WHEN comp_class = 'SORT'             THEN 1 END) AS
sort_cnt,
           COUNT(CASE WHEN comp_class = 'SPLIT'            THEN 1 END) AS
split_cnt,
           COUNT(CASE WHEN comp_class = 'DISTINCT'         THEN 1 END) AS
```

```
distinct_cnt,
           COUNT(CASE WHEN comp_class IN ('PIVOT', 'UNPIVOT') THEN 1 END) AS
pivot_cnt,
           COUNT(CASE WHEN comp_class = 'SUBQUERY_FILTER'  THEN 1 END) AS
subquery_filter_cnt,
           COUNT(CASE WHEN comp_class = 'REUSABLE_MAPPING' THEN 1 END) AS
reusable_mapping_cnt,
           COUNT(CASE WHEN comp_class = 'TABLE_FUNCTION'   THEN 1 END) AS
table_function_cnt,
           COUNT(CASE WHEN comp_class = 'DATASET'          THEN 1 END) AS
dataset_cnt,
           COUNT(CASE WHEN comp_class NOT IN ('DATASTORE', 'SIGNATURE',
'DATASET') THEN 1 END) AS transform_comp_cnt,
           LISTAGG(CASE WHEN comp_class = 'OTHER' THEN comp_type END, ', ' ON
OVERFLOW TRUNCATE '...' WITH COUNT)
               WITHIN GROUP (ORDER BY comp_type) AS other_types
    FROM   comp
    GROUP  BY comp.i_mapping
),
mpkg AS (
    SELECT i_mapping,
           COUNT(*) AS package_cnt,
           LISTAGG(pack_name, ', ' ON OVERFLOW TRUNCATE '...' WITH COUNT) WITHIN
GROUP (ORDER BY pack_name) AS packages
    FROM  (SELECT DISTINCT s.i_mapping, p.pack_name
           FROM   snp_step s
           JOIN   snp_package p ON p.i_package = s.i_package
           WHERE  s.i_mapping IS NOT NULL)                      -- VERIFY:
SNP_STEP.I_MAPPING
    GROUP  BY i_mapping
)
SELECT m.name                                          AS mapping_name,   -- VERIFY:
SNP_MAPPING.NAME (alt: MAPPING_NAME)
       pr.project_name,
       fld.folder_path,
       mp.package_cnt,
       mp.packages,
       mc.datastore_cnt, mc.join_cnt, mc.filter_cnt, mc.expression_cnt,
mc.aggregate_cnt,
       mc.lookup_cnt, mc.set_cnt, mc.sort_cnt, mc.split_cnt, mc.distinct_cnt,
mc.pivot_cnt,
       mc.subquery_filter_cnt, mc.reusable_mapping_cnt, mc.table_function_cnt,
mc.dataset_cnt,
       mc.transform_comp_cnt,
       mc.other_types,
       CASE
         WHEN NVL(mc.transform_comp_cnt, 0) = 0  THEN 'A copy (no transform
component)'
         WHEN mc.transform_comp_cnt <= 3         THEN 'B simple (1-3)'
         WHEN mc.transform_comp_cnt <= 9         THEN 'C medium (4-9)'
         ELSE                                         'D complex (10+)'
       END                                          AS complexity_band,
       CASE WHEN mp.package_cnt IS NULL THEN 'Y' END AS not_used_in_any_package,
       m.last_date
FROM   snp_mapping m
LEFT JOIN fld         ON fld.i_folder  = m.i_folder            -- VERIFY:
SNP_MAPPING.I_FOLDER
LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
LEFT JOIN mcnt     mc ON mc.i_mapping = m.i_mapping
LEFT JOIN mpkg     mp ON mp.i_mapping = m.i_mapping
ORDER  BY mc.transform_comp_cnt DESC NULLS LAST, m.name;
SPOOL OFF

-- 03b  KMs per package step, taken from the generated scenarios (design-time,
```

```
no run needed).
--      Scenario tasks name the KM or procedure in TASK_NAME1/2/3, the KM is the
one that
--      starts with LKM/IKM/CKM/JKM/RKM/SKM/XKM. The design-time physical-node
tables
--      (SNP_PHY_NODE etc.) differ by patch level and are not used, see 00c if
needed.
SPOOL odi_q03b_kms_by_scenario.csv
WITH t AS (
    SELECT st.scen_no,
           st.nno,
           COALESCE(
             CASE WHEN REGEXP_LIKE(st.task_name2, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name2 END,
             CASE WHEN REGEXP_LIKE(st.task_name1, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name1 END,
             CASE WHEN REGEXP_LIKE(st.task_name3, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name3 END
           ) AS km_name                                        -- VERIFY: which
TASK_NAMEn holds the KM name (usually TASK_NAME2)
    FROM   snp_scen_task st
)
SELECT p.pack_name,
       sc.scen_name,
       sc.scen_version,
       ss.step_name,                                           -- VERIFY:
SNP_SCEN_STEP (SCEN_NO, NNO, STEP_NAME)
       UPPER(SUBSTR(t.km_name, 1, 3))                      AS km_type,
       t.km_name,
       COUNT(*)                                            AS task_cnt
FROM   t
JOIN   snp_scen      sc ON sc.scen_no = t.scen_no
LEFT JOIN snp_package p  ON p.i_package = sc.i_package
LEFT JOIN snp_scen_step ss ON ss.scen_no = t.scen_no AND ss.nno = t.nno
WHERE  t.km_name IS NOT NULL
GROUP  BY p.pack_name, sc.scen_name, sc.scen_version, ss.step_name,
UPPER(SUBSTR(t.km_name, 1, 3)), t.km_name
ORDER  BY p.pack_name, ss.step_name, km_type;
SPOOL OFF

-- 03c  KM usage summary across the estate: from scenarios (all) and from
sessions in the
--      lookback window (what actually ran). Custom KMs here become Dataform
templates /
--      generic landing patterns.
SPOOL odi_q03c_km_summary.csv
WITH scen_km AS (
    SELECT COALESCE(
             CASE WHEN REGEXP_LIKE(st.task_name2, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name2 END,
             CASE WHEN REGEXP_LIKE(st.task_name1, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name1 END,
             CASE WHEN REGEXP_LIKE(st.task_name3, '^[LICJRSX]KM[ _]', 'i') THEN
st.task_name3 END
           ) AS km_name,
           st.scen_no
    FROM   snp_scen_task st
),
sess_km AS (
    SELECT COALESCE(
             CASE WHEN REGEXP_LIKE(tk.task_name2, '^[LICJRSX]KM[ _]', 'i') THEN
tk.task_name2 END,
             CASE WHEN REGEXP_LIKE(tk.task_name1, '^[LICJRSX]KM[ _]', 'i') THEN
tk.task_name1 END,
```

```
            CASE WHEN REGEXP_LIKE(tk.task_name3, '^[LICJRSX]KM[ _]', 'i') THEN
tk.task_name3 END
          ) AS km_name,                                    -- VERIFY:
SNP_SESS_TASK.TASK_NAME1/2/3 as in SNP_SCEN_TASK
          tk.sess_no
    FROM   snp_sess_task tk
    JOIN   snp_session   se ON se.sess_no = tk.sess_no
    WHERE  se.sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
)
SELECT NVL(a.km_type, b.km_type)             AS km_type,
       NVL(a.km_name, b.km_name)             AS km_name,
       a.scenario_cnt,
       b.session_cnt_in_window
FROM  (SELECT UPPER(SUBSTR(km_name, 1, 3)) km_type, km_name, COUNT(DISTINCT
scen_no) scenario_cnt
       FROM scen_km WHERE km_name IS NOT NULL GROUP BY UPPER(SUBSTR(km_name, 1,
3)), km_name) a
FULL OUTER JOIN
      (SELECT UPPER(SUBSTR(km_name, 1, 3)) km_type, km_name, COUNT(DISTINCT
sess_no) session_cnt_in_window
       FROM sess_km WHERE km_name IS NOT NULL GROUP BY UPPER(SUBSTR(km_name, 1,
3)), km_name) b
       ON b.km_name = a.km_name
ORDER  BY 1, 3 DESC NULLS LAST;
SPOOL OFF

--
================================================================================
=====
-- 04  PROCEDURES USED IN PACKAGES        (no form sheet, sizes the "mapping not
visible"
--     group, which is expected to be procedure / PL-SQL heavy)
--     Flags are text searches on the stored command text and can give false
positives,
--     they are for sizing, not for conversion. Full text from 04c can be fed to
the
--     BigQuery Migration Service batch SQL translator.
--
================================================================================
=====

-- 04a  One row per procedure.
SPOOL odi_q04a_procedures.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
ln AS (
    SELECT l.i_trt,
           l.ord_trt,
           l.sql_name                    AS command_name,
           l.def_tech_int_name           AS target_tech,      -- VERIFY:
DEF_TECH_INT_NAME (alt: I_DEF_TECHNO -> SNP_TECHNO)
           l.def_lschema_name            AS target_lschema,   -- VERIFY:
DEF_LSCHEMA_NAME
           l.col_tech_int_name           AS source_tech,      -- VERIFY:
```

```
COL_TECH_INT_NAME
           l.col_lschema_name            AS source_lschema,    -- VERIFY:
COL_LSCHEMA_NAME
           l.ind_err                     AS ignore_errors,     -- VERIFY: IND_ERR
(ignore-errors flag, alt: IND_IGNORE_ERR)
           td.full_text                  AS target_cmd,        -- "Command on
Target"
           tc.full_text                  AS source_cmd         -- "Command on
Source"
    FROM   snp_line_trt l
    LEFT JOIN snp_txtheader td ON td.i_txt = l.def_i_txt     -- VERIFY:
SNP_TXTHEADER.FULL_TEXT (12c), 11g: SNP_TXT chunks (I_TXT, TXT_ORD, TXT)
    LEFT JOIN snp_txtheader tc ON tc.i_txt = l.col_i_txt     -- VERIFY:
COL_I_TXT / DEF_I_TXT text pointers
    WHERE  l.i_trt IN (SELECT i_trt FROM snp_step WHERE i_trt IS NOT NULL)
)
,
fl AS (
    SELECT l.*,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|[^[:alnum:]_])(BEGIN|DECLARE)
([^[:alnum:]_]|$)', 'i')       THEN 1 ELSE 0 END AS f_plsql_block,
           CASE WHEN REGEXP_LIKE(l.target_cmd, 'EXECUTE[[:space:]]+IMMEDIATE|
DBMS_SQL', 'i')                   THEN 1 ELSE 0 END AS f_dynamic_sql,
           -- table@dblink, ODI substitution tags <@ ... @> are not matched, e-
mail addresses are (false positive).
           CASE WHEN REGEXP_LIKE(l.target_cmd, '[[:alnum:]_$#"]@[[:alpha:]"]')
                  OR REGEXP_LIKE(l.source_cmd, '[[:alnum:]_$#"]@[[:alpha:]"]')
THEN 1 ELSE 0 END AS f_db_link,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])CURSOR([^[:alnum:]_]|$)', 'i')               THEN 1 ELSE 0 END AS
f_cursor,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|[^[:alnum:]_])(LOOP|
FOR[[:space:]].*[[:space:]]IN)([^[:alnum:]_]|$)', 'i')

THEN 1 ELSE 0 END AS f_loop,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])MERGE[[:space:]]+INTO', 'i')                  THEN 1 ELSE 0 END AS
f_merge,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])TRUNCATE[[:space:]]', 'i')                    THEN 1 ELSE 0 END AS
f_truncate,
           CASE WHEN REGEXP_LIKE(l.target_cmd, 'UTL_FILE|UTL_HTTP|UTL_SMTP|
UTL_MAIL|DBMS_SCHEDULER|DBMS_JOB', 'i') THEN 1 ELSE 0 END AS f_external_io,
           CASE WHEN REGEXP_LIKE(l.target_cmd, 'odiRef\.', 'i')
THEN 1 ELSE 0 END AS f_odiref_generic,
           CASE WHEN l.source_cmd IS NOT NULL AND
DBMS_LOB.GETLENGTH(l.source_cmd) > 0                         THEN 1 ELSE 0 END
AS f_source_command,
           -- VERIFY: SQL technology internal names (MICROSOFT_SQL_SERVER,
MYSQL, POSTGRES_SQL), check with 12c / 07c
           CASE WHEN UPPER(NVL(l.target_tech, '?')) NOT IN ('ORACLE',
'MICROSOFT_SQL_SERVER', 'MYSQL', 'POSTGRES_SQL', 'POSTGRESQL', '?')

THEN 1 ELSE 0 END AS f_non_sql_tech
    FROM   ln l
),
lsum AS (
    SELECT i_trt,
           COUNT(*)                                                AS
line_cnt,
           SUM(NVL(DBMS_LOB.GETLENGTH(target_cmd), 0)
             + NVL(DBMS_LOB.GETLENGTH(source_cmd), 0))             AS
text_length,
           MAX(NVL(DBMS_LOB.GETLENGTH(target_cmd), 0))             AS
```

```
longest_line_length,
           SUM(f_plsql_block)    AS plsql_block_lines,
           SUM(f_dynamic_sql)    AS dynamic_sql_lines,
           SUM(f_db_link)        AS db_link_lines,
           SUM(f_cursor)         AS cursor_lines,
           SUM(f_loop)           AS loop_lines,
           SUM(f_merge)          AS merge_lines,
           SUM(f_truncate)       AS truncate_lines,
           SUM(f_external_io)    AS external_io_lines,
           SUM(f_odiref_generic) AS odiref_lines,
           SUM(f_source_command) AS source_command_lines,
           SUM(f_non_sql_tech)   AS non_sql_tech_lines,
           COUNT(CASE WHEN TO_CHAR(ignore_errors) IN ('1', 'Y') THEN 1 END) AS
ignore_error_lines
    FROM   fl
    GROUP  BY i_trt
),
ltech AS (
    SELECT i_trt, LISTAGG(tech, ', ') WITHIN GROUP (ORDER BY tech) AS
technologies
    FROM  (SELECT DISTINCT i_trt, target_tech AS tech FROM ln WHERE target_tech
IS NOT NULL
           UNION
           SELECT DISTINCT i_trt, source_tech FROM ln WHERE source_tech IS NOT
NULL)
    GROUP  BY i_trt
),
puse AS (
    SELECT i_trt,
           COUNT(*) AS package_cnt,
           LISTAGG(pack_name, ', ' ON OVERFLOW TRUNCATE '...' WITH COUNT) WITHIN
GROUP (ORDER BY pack_name) AS packages
    FROM  (SELECT DISTINCT s.i_trt, p.pack_name
           FROM   snp_step s JOIN snp_package p ON p.i_package = s.i_package
           WHERE  s.i_trt IS NOT NULL)
    GROUP  BY i_trt
)
SELECT t.trt_name                                    AS procedure_name,
       t.trt_type,                                   -- VERIFY: TRT_TYPE ('U' =
procedure, 'K' = KM, others)
       pr.project_name,
       fld.folder_path,
       pu.package_cnt,
       pu.packages,
       ls.line_cnt,
       ls.text_length,
       ls.longest_line_length,
       lt.technologies,
       ls.plsql_block_lines, ls.dynamic_sql_lines, ls.db_link_lines,
ls.cursor_lines, ls.loop_lines,
       ls.merge_lines, ls.truncate_lines, ls.external_io_lines, ls.odiref_lines,
       ls.source_command_lines, ls.non_sql_tech_lines, ls.ignore_error_lines,
       CASE
         WHEN ls.non_sql_tech_lines > 0
THEN 'Non-SQL logic (Jython / Groovy / OS / file) - rewrite'
         WHEN ls.dynamic_sql_lines > 0 OR ls.cursor_lines > 0 OR ls.loop_lines >
0

THEN 'Procedural PL/SQL - BigQuery scripting / stored procedure, manual'
         WHEN ls.plsql_block_lines > 0
THEN 'PL/SQL blocks - translate, then review'
         WHEN ls.source_command_lines > 0
THEN 'Source/target command pairs (data movement) - landing pattern'
         ELSE
```

```
'Plain SQL statements - batch SQL translation'
       END                                          AS conversion_hint,
       t.last_date
FROM   snp_trt t
JOIN   puse  pu ON pu.i_trt = t.i_trt
LEFT JOIN lsum ls ON ls.i_trt = t.i_trt
LEFT JOIN ltech lt ON lt.i_trt = t.i_trt
LEFT JOIN fld     ON fld.i_folder = t.i_folder               -- VERIFY:
SNP_TRT.I_FOLDER (project procedures, global procedures have none)
LEFT JOIN snp_project pr ON pr.i_project = NVL(fld.i_project, t.i_project)
ORDER  BY ls.text_length DESC NULLS LAST;
SPOOL OFF

-- 04b  One row per procedure line with flags (first 1000 characters of the
command).
SPOOL odi_q04b_procedure_lines.csv
WITH
ln AS (
    SELECT l.i_trt,
           l.ord_trt,
           l.sql_name                    AS command_name,
           l.def_tech_int_name           AS target_tech,      -- VERIFY:
DEF_TECH_INT_NAME (alt: I_DEF_TECHNO -> SNP_TECHNO)
           l.def_lschema_name            AS target_lschema,   -- VERIFY:
DEF_LSCHEMA_NAME
           l.col_tech_int_name           AS source_tech,      -- VERIFY:
COL_TECH_INT_NAME
           l.col_lschema_name            AS source_lschema,   -- VERIFY:
COL_LSCHEMA_NAME
           l.ind_err                     AS ignore_errors,    -- VERIFY: IND_ERR
(ignore-errors flag, alt: IND_IGNORE_ERR)
           td.full_text                  AS target_cmd,       -- "Command on
Target"
           tc.full_text                  AS source_cmd        -- "Command on
Source"
    FROM   snp_line_trt l
    LEFT JOIN snp_txtheader td ON td.i_txt = l.def_i_txt     -- VERIFY:
SNP_TXTHEADER.FULL_TEXT (12c), 11g: SNP_TXT chunks (I_TXT, TXT_ORD, TXT)
    LEFT JOIN snp_txtheader tc ON tc.i_txt = l.col_i_txt     -- VERIFY:
COL_I_TXT / DEF_I_TXT text pointers
    WHERE  l.i_trt IN (SELECT i_trt FROM snp_step WHERE i_trt IS NOT NULL)
)
,
fl AS (
    SELECT l.*,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|[^[:alnum:]_])(BEGIN|DECLARE)
([^[:alnum:]_]|$)', 'i')       THEN 1 ELSE 0 END AS f_plsql_block,
           CASE WHEN REGEXP_LIKE(l.target_cmd, 'EXECUTE[[:space:]]+IMMEDIATE|
DBMS_SQL', 'i')                   THEN 1 ELSE 0 END AS f_dynamic_sql,
           -- table@dblink, ODI substitution tags <@ ... @> are not matched, e-
mail addresses are (false positive).
           CASE WHEN REGEXP_LIKE(l.target_cmd, '[[:alnum:]_$#"]@[[:alpha:]"]')
                  OR REGEXP_LIKE(l.source_cmd, '[[:alnum:]_$#"]@[[:alpha:]"]')
THEN 1 ELSE 0 END AS f_db_link,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])CURSOR([^[:alnum:]_]|$)', 'i')               THEN 1 ELSE 0 END AS
f_cursor,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|[^[:alnum:]_])(LOOP|
FOR[[:space:]].*[[:space:]]IN)([^[:alnum:]_]|$)', 'i')

THEN 1 ELSE 0 END AS f_loop,
           CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])MERGE[[:space:]]+INTO', 'i')                 THEN 1 ELSE 0 END AS
f_merge,
```

```
          CASE WHEN REGEXP_LIKE(l.target_cmd, '(^|
[^[:alnum:]_])TRUNCATE[[:space:]]', 'i')                    THEN 1 ELSE 0 END AS
f_truncate,
          CASE WHEN REGEXP_LIKE(l.target_cmd, 'UTL_FILE|UTL_HTTP|UTL_SMTP|
UTL_MAIL|DBMS_SCHEDULER|DBMS_JOB', 'i') THEN 1 ELSE 0 END AS f_external_io,
          CASE WHEN REGEXP_LIKE(l.target_cmd, 'odiRef\.', 'i')
THEN 1 ELSE 0 END AS f_odiref_generic,
          CASE WHEN l.source_cmd IS NOT NULL AND
DBMS_LOB.GETLENGTH(l.source_cmd) > 0                         THEN 1 ELSE 0 END
AS f_source_command,
          -- VERIFY: SQL technology internal names (MICROSOFT_SQL_SERVER,
MYSQL, POSTGRES_SQL), check with 12c / 07c
          CASE WHEN UPPER(NVL(l.target_tech, '?')) NOT IN ('ORACLE',
'MICROSOFT_SQL_SERVER', 'MYSQL', 'POSTGRES_SQL', 'POSTGRESQL', '?')

THEN 1 ELSE 0 END AS f_non_sql_tech
    FROM   ln l
)
SELECT t.trt_name                                   AS procedure_name,
       fl.ord_trt                                   AS line_order,
       fl.command_name,
       fl.target_tech, fl.target_lschema, fl.source_tech, fl.source_lschema,
       fl.ignore_errors,
       DBMS_LOB.GETLENGTH(fl.target_cmd)            AS target_cmd_length,
       DBMS_LOB.GETLENGTH(fl.source_cmd)            AS source_cmd_length,
       fl.f_plsql_block, fl.f_dynamic_sql, fl.f_db_link, fl.f_cursor, fl.f_loop,
fl.f_merge,
       fl.f_truncate, fl.f_external_io, fl.f_odiref_generic,
fl.f_source_command, fl.f_non_sql_tech,
       DBMS_LOB.SUBSTR(fl.target_cmd, 1000, 1)    AS target_cmd_head
FROM   fl
JOIN   snp_trt t ON t.i_trt = fl.i_trt
ORDER  BY t.trt_name, fl.ord_trt;
SPOOL OFF

-- 04c  Full command text (CLOB) for offline analysis and SQL translation.
--      In SQLcl, SET LONG above controls how much of each CLOB is printed.
SPOOL odi_q04c_procedure_text.csv
SELECT t.trt_name                                   AS procedure_name,
       l.ord_trt                                    AS line_order,
       l.sql_name                                   AS command_name,
       l.def_tech_int_name                          AS target_tech,   -- VERIFY:
DEF_TECH_INT_NAME
       td.full_text                                 AS target_cmd,    -- VERIFY:
SNP_TXTHEADER.FULL_TEXT
       tc.full_text                                 AS source_cmd
FROM   snp_trt t
JOIN   snp_line_trt l ON l.i_trt = t.i_trt
LEFT JOIN snp_txtheader td ON td.i_txt = l.def_i_txt
LEFT JOIN snp_txtheader tc ON tc.i_txt = l.col_i_txt
WHERE  t.i_trt IN (SELECT i_trt FROM snp_step WHERE i_trt IS NOT NULL)
ORDER  BY t.trt_name, l.ord_trt;
SPOOL OFF

--
================================================================================
=====
-- 05  VARIABLES AND REFRESH QUERIES       (no form sheet, pattern: Variable ->
Airflow
--     variable / Dataform compilation variable). Many "evaluate" steps point to
loop
--     packages (e.g. the 5-minute loops in the schedule column).
--
================================================================================
```

```
=====

-- 05a  One row per variable with refresh query and package usage.
SPOOL odi_q05a_variables.csv
WITH vuse AS (
    SELECT s.i_var,
           COUNT(DISTINCT s.i_package)                              AS
package_cnt,
           COUNT(*)                                                 AS step_cnt,
           COUNT(CASE WHEN s.step_type = 'V'  THEN 1 END)           AS
refresh_steps,    -- VERIFY: variable STEP_TYPE codes V (refresh), VD (declare),
VS (set), VE (evaluate), see 00d-2
           COUNT(CASE WHEN s.step_type = 'VD' THEN 1 END)           AS
declare_steps,
           COUNT(CASE WHEN s.step_type = 'VS' THEN 1 END)           AS
set_steps,
           COUNT(CASE WHEN s.step_type = 'VE' THEN 1 END)           AS
evaluate_steps
    FROM   snp_step s
    WHERE  s.i_var IS NOT NULL
    GROUP  BY s.i_var
),
vpkg AS (
    SELECT i_var,
           LISTAGG(pack_name, ', ' ON OVERFLOW TRUNCATE '...' WITH COUNT) WITHIN
GROUP (ORDER BY pack_name) AS packages
    FROM  (SELECT DISTINCT s.i_var, p.pack_name
           FROM   snp_step s JOIN snp_package p ON p.i_package = s.i_package
           WHERE  s.i_var IS NOT NULL)
    GROUP  BY i_var
)
SELECT v.var_name,
       NVL(pr.project_name, '(global)')                AS scope,
       v.var_datatype,                                  -- VERIFY: VAR_DATATYPE
codes A/N/D/T
       v.ind_store                                      AS persistence,   --
VERIFY: IND_STORE (history / latest value / not persistent)
       v.def_v                                          AS default_value, --
VERIFY: DEF_V (alt: DEFAULT_VALUE)
       v.lschema_name                                   AS refresh_lschema,
       DBMS_LOB.GETLENGTH(tx.full_text)                 AS refresh_query_length,
       tx.full_text                                     AS refresh_query,
       vu.package_cnt, vu.step_cnt, vu.refresh_steps, vu.declare_steps,
vu.set_steps, vu.evaluate_steps,
       vp.packages,
       v.last_date
FROM   snp_var v
LEFT JOIN snp_project   pr ON pr.i_project = v.i_project
LEFT JOIN snp_txtheader tx ON tx.i_txt     = v.i_txt_var_in    -- VERIFY:
refresh-query pointer I_TXT_VAR_IN (alt: I_TXT_VAR, I_TXT_VAR_SEL), check 00c
LEFT JOIN vuse vu ON vu.i_var = v.i_var
LEFT JOIN vpkg vp ON vp.i_var = v.i_var
ORDER  BY vu.package_cnt DESC NULLS LAST, v.var_name;
SPOOL OFF

-- 05b  Variable references inside procedure text (#VAR, #PROJECT.VAR,
#GLOBAL.VAR).
--      Text search, can be slow on large repositories.
SPOOL odi_q05b_variable_refs_in_procedures.csv
SELECT v.var_name,
       COUNT(DISTINCT l.i_trt)                           AS procedure_cnt,
       COUNT(*)                                          AS line_cnt
FROM   snp_var v
CROSS JOIN snp_line_trt l
```

```
JOIN   snp_txtheader tx ON tx.i_txt = l.def_i_txt        -- VERIFY:
SNP_TXTHEADER.FULL_TEXT / DEF_I_TXT
WHERE  l.i_trt IN (SELECT i_trt FROM snp_step WHERE i_trt IS NOT NULL)
AND    DBMS_LOB.INSTR(tx.full_text, '#') > 0
AND    REGEXP_LIKE(tx.full_text, '#([[:alnum:]_]+\.)?' || v.var_name ||
'([^[:alnum:]_]|$)', 'i')
GROUP  BY v.var_name
ORDER  BY line_cnt DESC;
SPOOL OFF

--
================================================================================
=====
-- 06  JOURNALIZING / CDC                    (no form sheet, pattern:
Journalizing -> Datastream)
--     Evidence is collected from several places because the flag columns are
the least
--     certain part of the schema. Also see 11c: J$ tables in the DWH/source
databases.
--
================================================================================
=====

-- 06a  JKMs present in the repository (any JKM means CDC was at least set up).
SPOOL odi_q06a_jkms.csv
SELECT t.trt_name, pr.project_name, t.last_date
FROM   snp_trt t
LEFT JOIN snp_project pr ON pr.i_project = t.i_project
WHERE  UPPER(t.trt_name) LIKE 'JKM%'
ORDER  BY t.trt_name;
SPOOL OFF

-- 06b  Models with a JKM assigned and their journalized datastore count.
SPOOL odi_q06b_journalized_models.csv
SELECT md.cod_mod,
       md.mod_name,
       md.tech_int_name                                  AS technology,
       md.lschema_name,
       k.trt_name                                        AS jkm_name,
       COUNT(CASE WHEN TO_CHAR(t.ind_jrn) IN ('1', 'Y') THEN 1 END) AS
journalized_datastores,   -- VERIFY: SNP_TABLE.IND_JRN (alt: JRN_ORDER not null)
       COUNT(t.i_table)                                  AS datastores
FROM   snp_model md
LEFT JOIN snp_trt   k ON k.i_trt = md.i_trt_kjm          -- VERIFY:
SNP_MODEL.I_TRT_KJM (JKM of the model, alt via SNP_MOD_KM)
LEFT JOIN snp_table t ON t.i_mod = md.i_mod
GROUP  BY md.cod_mod, md.mod_name, md.tech_int_name, md.lschema_name, k.trt_name
HAVING k.trt_name IS NOT NULL
    OR COUNT(CASE WHEN TO_CHAR(t.ind_jrn) IN ('1', 'Y') THEN 1 END) > 0
ORDER  BY journalized_datastores DESC;
SPOOL OFF

-- 06c  Journalized datastores.
SPOOL odi_q06c_journalized_datastores.csv
SELECT md.cod_mod || '.' || t.table_name                 AS datastore,
       md.tech_int_name                                  AS technology,
       t.ind_jrn                                         -- VERIFY:
SNP_TABLE.IND_JRN
FROM   snp_table t
JOIN   snp_model md ON md.i_mod = t.i_mod
WHERE  TO_CHAR(t.ind_jrn) IN ('1', 'Y')
ORDER  BY 1;
SPOOL OFF
```

```
-- 06d  Package steps that start/consume journals, and scenario tasks that run a
JKM.
SPOOL odi_q06d_journalizing_steps.csv
SELECT p.pack_name,
       s.step_name,
       s.step_type,                                        -- VERIFY: journalizing
step codes JD (datastore) / JM (model)
       'package step'                                      AS evidence
FROM   snp_step s
JOIN   snp_package p ON p.i_package = s.i_package
WHERE  s.step_type IN ('JD', 'JM')
   OR  REGEXP_LIKE(s.step_name, 'JOURNAL|CDC|JRN', 'i')
UNION ALL
SELECT p.pack_name,
       NVL(st.task_name1, st.task_name2),
       NULL,
       'scenario task uses JKM: ' || NVL(st.task_name2, st.task_name3)
FROM   snp_scen_task st
JOIN   snp_scen sc ON sc.scen_no = st.scen_no
LEFT JOIN snp_package p ON p.i_package = sc.i_package
WHERE  REGEXP_LIKE(st.task_name1 || ' ' || st.task_name2 || ' ' ||
st.task_name3, '(^|[^[:alnum:]])JKM[ _]', 'i')
ORDER  BY 1, 2;
SPOOL OFF

--
================================================================================
=====
-- 07  FILE AND NON-DATABASE SOURCES         (no form sheet, pattern: File LKM -
> GCS +
--     BigQuery load). Technology names are ODI internal names (TECH_INT_NAME).
--
================================================================================
=====

-- 07a  Non-database datastores used by package mappings.
SPOOL odi_q07a_non_db_datastores.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
pkg AS (
    SELECT p.i_package,
           p.pack_name,
           pr.project_name,
           fld.folder_path,
           p.last_date,
           p.last_user
    FROM   snp_package p
    LEFT JOIN fld         ON fld.i_folder  = p.i_folder
    LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
```

```
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
             WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
             WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
             WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
             WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
             WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
             WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
             WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
             WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
             WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
             WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
             WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
             WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
             WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
             WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
             WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
             WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
             ELSE 'OTHER'
           END                         AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
SOURCE
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                     SELECT 1
                     FROM   snp_map_cp   cp
                     JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                     WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
```

```
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
)
SELECT pkg.pack_name,
       pds.ds_role,
       pds.ds_name,
       pds.technology,
       pds.lschema_name
FROM   pds
JOIN   pkg ON pkg.i_package = pds.i_package
WHERE  REGEXP_LIKE(NVL(pds.technology, '?'),
                   'FILE|XML|JMS|JSON|LDAP|HDFS|HBASE|HIVE|SPARK|KAFKA|REST|
SOAP|SALESFORCE|ESSBASE|HYPERION|PLANNING|OLAP|BAM|EXCEL|MAIL|^\?$',
                   'i')                                      -- '?' = technology
not resolved, listed for review
ORDER  BY pkg.pack_name, pds.ds_role, pds.ds_name;
SPOOL OFF

-- 07b  Technologies and KMs per package as generated in its scenario: covers
mapping,
--      procedure and tool steps (FILE, Jython/Groovy, ODI tools, OS commands).
SPOOL odi_q07b_scenario_technologies.csv
WITH t AS (
    SELECT p.pack_name,
           sc.scen_name,
           NVL(st.def_tech_int_name, '-')               AS target_tech,   --
VERIFY: SNP_SCEN_TASK.DEF_TECH_INT_NAME
           NVL(st.col_tech_int_name, '-')               AS source_tech,   --
VERIFY: SNP_SCEN_TASK.COL_TECH_INT_NAME
           st.task_name2                                AS km_or_procedure,
           COUNT(*)                                     AS task_cnt
    FROM   snp_scen_task st
    JOIN   snp_scen sc ON sc.scen_no = st.scen_no
    LEFT JOIN snp_package p ON p.i_package = sc.i_package
    GROUP  BY p.pack_name, sc.scen_name, NVL(st.def_tech_int_name, '-'),
NVL(st.col_tech_int_name, '-'), st.task_name2
)
SELECT pack_name,
       scen_name,
       target_tech,
       source_tech,
```

```
       SUM(task_cnt)                                        AS task_cnt,
       LISTAGG(km_or_procedure, ', ' ON OVERFLOW TRUNCATE '...' WITH COUNT)
           WITHIN GROUP (ORDER BY km_or_procedure)       AS
km_or_procedure_names
FROM   t
WHERE  REGEXP_LIKE(target_tech || ' ' || source_tech,
                   'FILE|XML|JMS|JSON|LDAP|HDFS|HIVE|KAFKA|REST|SOAP|SALESFORCE|
ESSBASE|JYTHON|GROOVY|JAVA|OS|SUNOPSIS_API|ODI', 'i')   -- VERIFY: internal
names of file / script / tool technologies (compare with 12c and 07c)
GROUP  BY pack_name, scen_name, target_tech, source_tech
ORDER  BY pack_name, task_cnt DESC;
SPOOL OFF

-- 07c  Topology (master repository tables, same schema in 12c): which physical
server and
--      schema each logical schema points to, per context. Counts the real
source systems
--      and gives the physical schema names for TARGET_SCHEMAS in query 11.
SPOOL odi_q07c_topology.csv
SELECT ctx.context_code,                                    -- VERIFY: SNP_CONTEXT
(I_CONTEXT, CONTEXT_CODE)
       te.tech_int_name                                     AS technology,  --
VERIFY: SNP_TECHNO (I_TECHNO, TECH_INT_NAME)
       ls.lschema_name                                      AS logical_schema,
       cn.con_name                                          AS data_server, --
VERIFY: SNP_CONNECT (I_CONNECT, CON_NAME, I_TECHNO)
       ps.schema_name                                       AS physical_schema,  --
VERIFY: SNP_PSCHEMA (I_PSCHEMA, I_CONNECT, SCHEMA_NAME)
       (SELECT COUNT(*) FROM snp_model md WHERE md.lschema_name =
ls.lschema_name) AS model_cnt
FROM   snp_lschema ls                                       -- VERIFY: SNP_LSCHEMA
(I_LSCHEMA, LSCHEMA_NAME)
LEFT JOIN snp_pschema_cont pc  ON pc.i_lschema  = ls.i_lschema   -- VERIFY:
SNP_PSCHEMA_CONT (I_LSCHEMA, I_PSCHEMA, I_CONTEXT)
LEFT JOIN snp_pschema      ps  ON ps.i_pschema  = pc.i_pschema
LEFT JOIN snp_connect      cn  ON cn.i_connect  = ps.i_connect
LEFT JOIN snp_techno       te  ON te.i_techno   = cn.i_techno
LEFT JOIN snp_context      ctx ON ctx.i_context = pc.i_context
ORDER  BY ctx.context_code, te.tech_int_name, ls.lschema_name;
SPOOL OFF

--
================================================================================
=====
-- 08  LOAD PLANS AND SCHEDULES       -> sheet 3 "Schedule" column, sheet 6
dependencies.
--     Pattern: Load Plan -> Composer DAG, serial/parallel step -> task group.
--
================================================================================
=====

-- 08a  Load plan step tree (serial / parallel / scenario), in execution order.
SPOOL odi_q08a_load_plan_tree.csv
WITH tree AS (
    SELECT lp.load_plan_name,
           LEVEL                                            AS depth,
           LPAD(' ', 2 * (LEVEL - 1)) || ls.lp_step_name   AS step_tree,
           SYS_CONNECT_BY_PATH(REPLACE(ls.lp_step_name, '>', '-'), ' > ') AS
step_path,
           ls.lp_step_type,                                 -- VERIFY:
LP_STEP_TYPE codes, see 00d-4
           ls.scen_name,
           ls.scen_version,                                 -- '-1' usually means
"latest version"
```

```
           ls.ind_enabled,
           ls.restart_type                                  -- VERIFY:
RESTART_TYPE (alt: RESTART_TYP)
    FROM   snp_lp_step ls
    JOIN   snp_load_plan lp ON lp.i_load_plan = ls.i_load_plan
    START WITH ls.par_i_lp_step IS NULL                     -- VERIFY: parent
column PAR_I_LP_STEP
    CONNECT BY PRIOR ls.i_lp_step = ls.par_i_lp_step
    ORDER SIBLINGS BY ls.step_order                         -- VERIFY: STEP_ORDER
(alt: LP_STEP_ORDER)
),
tree_ord AS (
    -- ROWNUM over the ordered tree keeps the execution order after the outer
join.
    SELECT x.*, ROWNUM AS ord FROM tree x
),
spkg AS (
    SELECT sc.scen_name, MAX(p.pack_name) AS pack_name
    FROM   snp_scen sc JOIN snp_package p ON p.i_package = sc.i_package
    GROUP  BY sc.scen_name
)
SELECT t.load_plan_name,
       t.depth,
       t.step_tree,
       t.lp_step_type,
       CASE t.lp_step_type
         WHEN 'SE' THEN 'Serial'
         WHEN 'PA' THEN 'Parallel'
         WHEN 'RS' THEN 'Run scenario'
         WHEN 'CS' THEN 'Case'
         WHEN 'CW' THEN 'When'
         WHEN 'CE' THEN 'Else'
         ELSE 'Other (' || t.lp_step_type || ')'
       END                                                  AS step_kind,   --
VERIFY: code meanings
       t.scen_name,
       t.scen_version,
       sp.pack_name,
       t.ind_enabled,
       t.restart_type,
       t.step_path
FROM   tree_ord t
LEFT JOIN spkg sp ON sp.scen_name = t.scen_name
ORDER  BY t.ord;
SPOOL OFF

-- 08b  Raw schedule table (all columns, safe on every version). Use it to
confirm the
--      column names used in 01 and 08c.
SPOOL odi_q08b_schedules_raw.csv
SELECT pa.*
FROM   snp_plan_agent pa;
SPOOL OFF

-- 08c  Schedules, readable.
SPOOL odi_q08c_schedules.csv
SELECT NVL(lp.load_plan_name, pa.scen_name)              AS scheduled_object,
       CASE WHEN lp.load_plan_name IS NOT NULL THEN 'Load plan' ELSE 'Scenario'
END AS object_type,
       pa.scen_version,
       pa.agent_name,
       pa.stat_plan                                      AS status,       --
VERIFY: STAT_PLAN codes
       pa.s_type                                         AS repeat_type,  --
```

```
VERIFY: S_TYPE (alt: EXEC_TYPE / PLAN_TYPE)
       LPAD(TO_CHAR(pa.s_hour), 2, '0') || ':' || LPAD(TO_CHAR(pa.s_minute), 2,
'0') AS start_time   -- VERIFY: S_HOUR / S_MINUTE
FROM   snp_plan_agent pa
LEFT JOIN snp_load_plan lp ON lp.i_load_plan = pa.i_load_plan            --
VERIFY: SNP_PLAN_AGENT.I_LOAD_PLAN
ORDER  BY start_time, scheduled_object;
SPOOL OFF

-- 08d  Load plan summary: size, parallelism, disabled steps.
SPOOL odi_q08d_load_plan_summary.csv
WITH tree AS (
    SELECT ls.i_load_plan, ls.lp_step_type, ls.scen_name, ls.ind_enabled, LEVEL
AS depth
    FROM   snp_lp_step ls
    START WITH ls.par_i_lp_step IS NULL
    CONNECT BY PRIOR ls.i_lp_step = ls.par_i_lp_step
)
SELECT lp.load_plan_name,
       COUNT(*)                                                          AS step_cnt,
       COUNT(CASE WHEN t.scen_name IS NOT NULL THEN 1 END)       AS
scenario_steps,
       COUNT(DISTINCT t.scen_name)                                       AS
distinct_scenarios,
       COUNT(CASE WHEN t.lp_step_type = 'PA' THEN 1 END)         AS
parallel_nodes,   -- VERIFY: 'PA'
       COUNT(CASE WHEN t.lp_step_type = 'SE' THEN 1 END)         AS
serial_nodes,     -- VERIFY: 'SE'
       COUNT(CASE WHEN TO_CHAR(t.ind_enabled) IN ('0', 'N') THEN 1 END) AS
disabled_steps,
       MAX(t.depth)                                                      AS max_depth,
       lp.last_date
FROM   tree t
JOIN   snp_load_plan lp ON lp.i_load_plan = t.i_load_plan
GROUP  BY lp.load_plan_name, lp.last_date
ORDER  BY step_cnt DESC;
SPOOL OFF

--
================================================================================
=====
-- 09  RUNTIME PER SCENARIO                      -> sheet "5 - Performans & SLA"
--     # = ROW_NO, Job Adi = JOB_NAME, Min/Ort./Max Sure (dk) =
MIN/AVG/MAX_DURATION_MIN,
--     Basari Orani (%) = SUCCESS_RATE_PCT, Hata Sayisi = ERROR_CNT,
--     En Sik Hata = MOST_FREQUENT_ERROR (LAST_ERROR also given).
--     SLA (deadline) is not in the repository: agree it with the customer,
using the
--     load plan end times from 09b.
--     Status codes: D = done, E = error, M = warning, R = running, W = waiting,
Q = queued.
--     Warnings count as success. Running/waiting sessions are excluded.
--
================================================================================
=====
SPOOL odi_q09a_runtime_per_scenario.csv
WITH s AS (
    SELECT NVL(se.scen_name, se.sess_name)                                AS
scen_name,   -- VERIFY: SNP_SESSION.SCEN_NAME (alt: SESS_NAME only)
           se.sess_no,
           se.sess_beg,
           NVL(se.sess_dur,
-- VERIFY: SESS_DUR in seconds
               (CAST(se.sess_end AS DATE) - CAST(se.sess_beg AS DATE)) *
```

```
86400) / 60 AS dur_min,
           se.sess_status,
-- VERIFY: status codes, 00d-4
           DBMS_LOB.SUBSTR(se.error_message, 1000, 1)                          AS
err_txt      -- VERIFY: ERROR_MESSAGE type (00b DATA_TYPE). CLOB in 12c. If
VARCHAR2 and DBMS_LOB.SUBSTR errors, use SUBSTR(se.error_message, 1, 1000)
    FROM   snp_session se
    WHERE  se.sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
    AND    se.sess_status IN ('D', 'E', 'M')
),
e AS (
    -- Error key: the first ORA- line if present, else the start of the message.
    SELECT s.*,
           NVL(SUBSTR(REGEXP_SUBSTR(s.err_txt, 'ORA-[0-9]{5}:[^' || CHR(10) ||
CHR(13) || ']*'), 1, 200),
               SUBSTR(REPLACE(REPLACE(s.err_txt, CHR(10), ' '), CHR(13), ' '),
1, 200)) AS err_key
    FROM   s
    WHERE  s.sess_status = 'E'
),
agg AS (
    SELECT scen_name,
           COUNT(*)                                                            AS
runs,
           MIN(sess_beg)                                                       AS
first_run_in_window,
           MAX(sess_beg)                                                       AS
last_run,
           ROUND(MIN(dur_min), 1)                                              AS
min_duration_min,
           ROUND(AVG(dur_min), 1)                                              AS
avg_duration_min,
           ROUND(MEDIAN(dur_min), 1)                                           AS
median_duration_min,
           ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY dur_min), 1)  AS
p95_duration_min,
           ROUND(MAX(dur_min), 1)                                              AS
max_duration_min,
           COUNT(CASE WHEN sess_status = 'E' THEN 1 END)                       AS
error_cnt,
           COUNT(CASE WHEN sess_status = 'M' THEN 1 END)                       AS
warning_cnt,
           ROUND(100 * COUNT(CASE WHEN sess_status IN ('D', 'M') THEN 1 END) /
COUNT(*), 1) AS success_rate_pct,
           COUNT(DISTINCT TRUNC(sess_beg))                                     AS
run_days
    FROM   s
    GROUP  BY scen_name
),
lasterr AS (
    SELECT scen_name, err_key AS last_error, sess_beg AS last_error_date
    FROM  (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY scen_name ORDER BY
sess_beg DESC) rn FROM e)
    WHERE  rn = 1
),
toperr AS (
    SELECT scen_name, err_key AS most_frequent_error, cnt AS
most_frequent_error_cnt
    FROM  (SELECT scen_name, err_key, COUNT(*) cnt,
                  ROW_NUMBER() OVER (PARTITION BY scen_name ORDER BY COUNT(*)
DESC, err_key) rn
           FROM   e
           GROUP  BY scen_name, err_key)
    WHERE  rn = 1
```

```
),
spkg AS (
    SELECT sc.scen_name, MAX(p.pack_name) AS pack_name
    FROM   snp_scen sc JOIN snp_package p ON p.i_package = sc.i_package
    GROUP  BY sc.scen_name
)
SELECT ROW_NUMBER() OVER (ORDER BY NVL(sp.pack_name, a.scen_name)) AS row_no,
-- sheet 5: #
       NVL(sp.pack_name, a.scen_name)                                AS job_name,
-- sheet 5: Job Adi
       a.scen_name,
       a.runs,
       a.run_days,
       a.first_run_in_window,
       a.last_run,
       a.min_duration_min,
-- sheet 5: Min Sure (dk)
       a.avg_duration_min,
-- sheet 5: Ort. Sure (dk), also sheet 3 Ort. Sure
       a.median_duration_min,
       a.p95_duration_min,
       a.max_duration_min,
-- sheet 5: Max Sure (dk)
       a.success_rate_pct,
-- sheet 5: Basari Orani (%)
       a.error_cnt,
-- sheet 5: Hata Sayisi
       a.warning_cnt,
       CAST(NULL AS VARCHAR2(20))                                    AS
sla_deadline, -- sheet 5: SLA (manual)
       t.most_frequent_error,
-- sheet 5: En Sik Hata
       t.most_frequent_error_cnt,
       l.last_error,
       l.last_error_date
FROM   agg a
LEFT JOIN spkg    sp ON sp.scen_name = a.scen_name
LEFT JOIN toperr  t  ON t.scen_name  = a.scen_name
LEFT JOIN lasterr l  ON l.scen_name  = a.scen_name
ORDER  BY job_name;
SPOOL OFF

-- 09b  Load plan runs: start/end time of day and duration -> the nightly batch
window,
--      which is the basis for the SLA column and for Composer DAG timing.
SPOOL odi_q09b_load_plan_runs.csv
WITH r AS (
    SELECT li.load_plan_name,                                    -- VERIFY:
SNP_LP_INST.LOAD_PLAN_NAME (alt: join SNP_LOAD_PLAN via I_LOAD_PLAN)
           r.start_date,                                         -- VERIFY:
SNP_LPI_RUN START_DATE / END_DATE / STATUS
           r.end_date,
           r.status,
           (CAST(r.end_date AS DATE) - CAST(r.start_date AS DATE)) * 1440
AS dur_min,
           (CAST(r.start_date AS DATE) - TRUNC(CAST(r.start_date AS DATE))) *
1440   AS start_min_of_day,
           (CAST(r.end_date AS DATE)   - TRUNC(CAST(r.start_date AS DATE))) *
1440   AS end_min_from_start_day,
           DBMS_LOB.SUBSTR(r.error_message, 500, 1)
AS err_txt
    FROM   snp_lpi_run r
    JOIN   snp_lp_inst li ON li.i_lp_inst = r.i_lp_inst
    WHERE  r.start_date >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
```

```
)
SELECT load_plan_name,
       COUNT(*)                                                    AS runs,
       COUNT(CASE WHEN status = 'E' THEN 1 END)                    AS
error_runs,     -- VERIFY: status code 'E'
       ROUND(AVG(dur_min), 1)                                      AS
avg_duration_min,
       ROUND(MAX(dur_min), 1)                                      AS
max_duration_min,
       TO_CHAR(TRUNC(SYSDATE) + AVG(start_min_of_day) / 1440, 'HH24:MI')
AS typical_start,
       TO_CHAR(TRUNC(SYSDATE) + AVG(end_min_from_start_day) / 1440, 'HH24:MI')
AS avg_end_time,
       TO_CHAR(TRUNC(SYSDATE) + PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY
end_min_from_start_day) / 1440, 'HH24:MI') AS p95_end_time,
       TO_CHAR(TRUNC(SYSDATE) + MAX(end_min_from_start_day) / 1440, 'HH24:MI')
AS latest_end_time,  -- past midnight shows as next-day clock time
       MAX(CASE WHEN status = 'E' THEN err_txt END)                 AS
an_error_message
FROM   r
GROUP  BY load_plan_name
ORDER  BY runs DESC;
SPOOL OFF

--
================================================================================
=====
-- 10  DAILY VOLUME PER SCENARIO        -> sheet 2 / 4 volume gaps (form had no
row counts),
--     sizes Datastream throughput and BigQuery slot needs.
--     NB_INS/NB_UPD/NB_DEL are filled only by KM/procedure tasks with a log
counter, so
--     they approximate rows written to targets. NB_ROW is filled by most tasks,
including
--     staging (C$/I$) loads, so it over-counts, it is shown as "rows
processed".
--     If the session log level was low, task rows may be purged after success,
10c then
--     gives the step-level fallback.
--
================================================================================
=====

-- 10a  Per scenario: average and max per day over the window.
SPOOL odi_q10a_daily_volume.csv
WITH d AS (
    SELECT NVL(se.scen_name, se.sess_name)       AS scen_name,     -- VERIFY:
SNP_SESSION.SCEN_NAME
           TRUNC(se.sess_beg)                   AS run_day,
           SUM(NVL(tl.nb_row, 0))               AS rows_processed,  -- VERIFY:
SNP_SESS_TASK_LOG NB_ROW/NB_INS/NB_UPD/NB_DEL/NB_ERR
           SUM(NVL(tl.nb_ins, 0))               AS rows_inserted,
           SUM(NVL(tl.nb_upd, 0))               AS rows_updated,
           SUM(NVL(tl.nb_del, 0))               AS rows_deleted,
           SUM(NVL(tl.nb_err, 0))               AS rows_error
    FROM   snp_sess_task_log tl
    JOIN   snp_session       se ON se.sess_no = tl.sess_no
    WHERE  se.sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
    GROUP  BY NVL(se.scen_name, se.sess_name), TRUNC(se.sess_beg)
),
spkg AS (
    SELECT sc.scen_name, MAX(p.pack_name) AS pack_name
    FROM   snp_scen sc JOIN snp_package p ON p.i_package = sc.i_package
    GROUP  BY sc.scen_name
```

```
)
SELECT NVL(sp.pack_name, d.scen_name)                          AS job_name,
       d.scen_name,
       COUNT(*)                                                AS days_with_runs,
       ROUND(AVG(d.rows_inserted + d.rows_updated + d.rows_deleted)) AS
avg_rows_written_per_day,
       MAX(d.rows_inserted + d.rows_updated + d.rows_deleted)        AS
max_rows_written_per_day,
       ROUND(AVG(d.rows_inserted))                             AS
avg_inserts_per_day,
       MAX(d.rows_inserted)                                    AS
max_inserts_per_day,
       ROUND(AVG(d.rows_updated))                              AS
avg_updates_per_day,
       MAX(d.rows_updated)                                     AS
max_updates_per_day,
       ROUND(AVG(d.rows_deleted))                              AS
avg_deletes_per_day,
       MAX(d.rows_deleted)                                     AS
max_deletes_per_day,
       ROUND(AVG(d.rows_processed))                            AS
avg_rows_processed_per_day,
       MAX(d.rows_processed)                                   AS
max_rows_processed_per_day,
       SUM(d.rows_error)                                       AS
rows_error_total
FROM   d
LEFT JOIN spkg sp ON sp.scen_name = d.scen_name
GROUP  BY NVL(sp.pack_name, d.scen_name), d.scen_name
ORDER  BY max_rows_written_per_day DESC;
SPOOL OFF

-- 10b  Per scenario step (usually one mapping = one target): average rows
written per day.
SPOOL odi_q10b_daily_volume_per_step.csv
WITH d AS (
    SELECT NVL(se.scen_name, se.sess_name)       AS scen_name,
           ss.step_name,                                          -- VERIFY:
SNP_SESS_STEP (SESS_NO, NNO, STEP_NAME)
           TRUNC(se.sess_beg)                   AS run_day,
           SUM(NVL(tl.nb_ins, 0) + NVL(tl.nb_upd, 0) + NVL(tl.nb_del, 0)) AS
rows_written
    FROM   snp_sess_task_log tl
    JOIN   snp_session       se ON se.sess_no = tl.sess_no
    LEFT JOIN snp_sess_step  ss ON ss.sess_no = tl.sess_no AND ss.nno = tl.nno
    WHERE  se.sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
    GROUP  BY NVL(se.scen_name, se.sess_name), ss.step_name, TRUNC(se.sess_beg)
)
SELECT scen_name,
       step_name,
       COUNT(*)                     AS days_with_runs,
       ROUND(AVG(rows_written))     AS avg_rows_written_per_day,
       MAX(rows_written)            AS max_rows_written_per_day
FROM   d
GROUP  BY scen_name, step_name
HAVING MAX(rows_written) > 0
ORDER  BY max_rows_written_per_day DESC;
SPOOL OFF

-- 10c  Fallback when task logs are purged: step-level counters.
SPOOL odi_q10c_daily_volume_step_log.csv
WITH d AS (
    SELECT NVL(se.scen_name, se.sess_name)       AS scen_name,
           TRUNC(se.sess_beg)                   AS run_day,
```

```
           SUM(NVL(sl.nb_ins, 0) + NVL(sl.nb_upd, 0) + NVL(sl.nb_del, 0)) AS
rows_written,   -- VERIFY: SNP_STEP_LOG NB_INS/NB_UPD/NB_DEL/NB_ROW
           SUM(NVL(sl.nb_row, 0))                AS rows_processed
    FROM   snp_step_log sl
    JOIN   snp_session  se ON se.sess_no = sl.sess_no
    WHERE  se.sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS
    GROUP  BY NVL(se.scen_name, se.sess_name), TRUNC(se.sess_beg)
)
SELECT scen_name,
       COUNT(*)                     AS days_with_runs,
       ROUND(AVG(rows_written))     AS avg_rows_written_per_day,
       MAX(rows_written)            AS max_rows_written_per_day,
       ROUND(AVG(rows_processed))   AS avg_rows_processed_per_day,
       MAX(rows_processed)          AS max_rows_processed_per_day
FROM   d
GROUP  BY scen_name
ORDER  BY max_rows_written_per_day DESC;
SPOOL OFF

--
================================================================================
=====
-- 11  TARGET DB DICTIONARY            *** RUN ON THE DWH DATABASE, NOT THE
REPOSITORY ***
--     -> sheet "2 - Veri Envanteri" (row counts, sizes), BigQuery storage
sizing, and
--        the ODS -> DWH_STG -> DWH -> DM layer picture.
--     Uses Oracle dictionary views only. Note: there is no ALL_SEGMENTS view,
sizes need
--     DBA_SEGMENTS (SELECT_CATALOG_ROLE or SELECT ANY DICTIONARY). Without it,
run 11a
--     per schema as that schema's user with USER_SEGMENTS (drop the owner
filter), or
--     use BLOCKS * block size from ALL_TABLES as an estimate.
--     NUM_ROWS is the optimizer estimate as of LAST_ANALYZED, not a live count.
--     TARGET_SCHEMAS: physical schema names (see 07c), e.g.
'ODS','DWH_STG','DWH','DM'.
--
================================================================================
=====

-- 11a  Per table: rows, statistics date, table / index / LOB size in GB.
SPOOL odi_q11a_dwh_tables.csv
WITH seg AS (
    SELECT owner, segment_name, segment_type, SUM(bytes) AS bytes
    FROM   dba_segments                                   -- VERIFY:
privilege, alt USER_SEGMENTS per schema
    WHERE  owner IN (&TARGET_SCHEMAS)
    GROUP  BY owner, segment_name, segment_type
),
tseg AS (
    SELECT owner, segment_name AS table_name, SUM(bytes) AS bytes
    FROM   seg
    WHERE  segment_type LIKE 'TABLE%'
    GROUP  BY owner, segment_name
),
iseg AS (
    SELECT i.table_owner AS owner, i.table_name, SUM(s.bytes) AS bytes
    FROM   dba_indexes i
    JOIN   seg s ON s.owner = i.owner AND s.segment_name = i.index_name AND
s.segment_type LIKE 'INDEX%'
    WHERE  i.table_owner IN (&TARGET_SCHEMAS)
    GROUP  BY i.table_owner, i.table_name
),
```

```
lseg AS (
    SELECT l.owner, l.table_name, SUM(s.bytes) AS bytes
    FROM   dba_lobs l
    JOIN   seg s ON s.owner = l.owner
              AND s.segment_name IN (l.segment_name, l.index_name)
              AND (s.segment_type LIKE 'LOB%')
    WHERE  l.owner IN (&TARGET_SCHEMAS)
    GROUP  BY l.owner, l.table_name
)
SELECT t.owner                                              AS schema_layer,
       t.table_name,
       t.num_rows,
       t.last_analyzed,
       t.partitioned,
       t.compression,
       ROUND(NVL(ts.bytes, 0) / POWER(1024, 3), 3)          AS table_gb,
       ROUND(NVL(ix.bytes, 0) / POWER(1024, 3), 3)          AS index_gb,
       ROUND(NVL(lb.bytes, 0) / POWER(1024, 3), 3)          AS lob_gb,
       ROUND((NVL(ts.bytes, 0) + NVL(ix.bytes, 0) + NVL(lb.bytes, 0)) /
POWER(1024, 3), 3) AS total_gb,
       CASE
         WHEN REGEXP_LIKE(t.table_name, '^(C|I|E|J|JV|T)\$') THEN 'ODI work
table'
         WHEN REGEXP_LIKE(t.table_name, 'TEST|_TMP|_BCK|_BAK|_OLD|YEDEK', 'i')
THEN 'Test / backup name'
       END                                                  AS table_note
FROM   all_tables t
LEFT JOIN tseg ts ON ts.owner = t.owner AND ts.table_name = t.table_name
LEFT JOIN iseg ix ON ix.owner = t.owner AND ix.table_name = t.table_name
LEFT JOIN lseg lb ON lb.owner = t.owner AND lb.table_name = t.table_name
WHERE  t.owner IN (&TARGET_SCHEMAS)
AND    t.temporary = 'N'
AND    NVL(t.dropped, 'NO') = 'NO'
ORDER  BY total_gb DESC, t.owner, t.table_name;
SPOOL OFF

-- 11b  Per layer (schema): table count, rows, size, statistics health, work
tables.
SPOOL odi_q11b_dwh_layers.csv
WITH seg AS (
    SELECT owner, SUM(bytes) AS bytes,
           SUM(CASE WHEN segment_type LIKE 'TABLE%' THEN bytes END) AS
table_bytes
    FROM   dba_segments                                      -- VERIFY: privilege
    WHERE  owner IN (&TARGET_SCHEMAS)
    GROUP  BY owner
)
SELECT t.owner                                                      AS
schema_layer,
       COUNT(*)                                                     AS
table_cnt,
       SUM(t.num_rows)                                              AS
total_rows,
       COUNT(CASE WHEN t.last_analyzed IS NULL THEN 1 END)          AS
never_analyzed,
       COUNT(CASE WHEN t.last_analyzed < SYSDATE - 30 THEN 1 END)   AS
stats_older_30d,
       COUNT(CASE WHEN t.partitioned = 'YES' THEN 1 END)            AS
partitioned_tables,
       COUNT(CASE WHEN REGEXP_LIKE(t.table_name, '^(C|I|E|J|JV|T)\$') THEN 1
END) AS odi_work_tables,
       ROUND(MAX(s.table_bytes) / POWER(1024, 3), 2)                AS
table_gb,
       ROUND(MAX(s.bytes) / POWER(1024, 3), 2)                      AS
```

```
all_segments_gb
FROM   all_tables t
LEFT JOIN seg s ON s.owner = t.owner
WHERE  t.owner IN (&TARGET_SCHEMAS)
AND    t.temporary = 'N'
AND    NVL(t.dropped, 'NO') = 'NO'
GROUP  BY t.owner
ORDER  BY t.owner;
SPOOL OFF

-- 11c  ODI work tables left in the database by KMs: C$ (LKM staging), I$ (IKM
flow),
--      E$ (CKM errors), J$ / JV$ (journalizing = CDC in use).
SPOOL odi_q11c_odi_work_tables.csv
SELECT owner,
       REGEXP_SUBSTR(table_name, '^(C|I|E|J|JV|T)\$')     AS prefix,
       COUNT(*)                                            AS table_cnt,
       SUM(num_rows)                                       AS total_rows,
       MIN(table_name)                                     AS example_1,
       MAX(table_name)                                     AS example_2
FROM   all_tables
WHERE  REGEXP_LIKE(table_name, '^(C|I|E|J|JV|T)\$')
AND    owner NOT IN ('SYS', 'SYSTEM')
GROUP  BY owner, REGEXP_SUBSTR(table_name, '^(C|I|E|J|JV|T)\$')
ORDER  BY owner, prefix;
SPOOL OFF

-- 11d  DML activity since the last statistics gather (inserts/updates/deletes
per table).
--      Monitoring data is flushed periodically by Oracle, so recent activity
may lag,
--      this script does not flush it (that would be a write).
SPOOL odi_q11d_dwh_dml_activity.csv
SELECT m.table_owner                                       AS schema_layer,
       m.table_name,
       SUM(m.inserts)                                      AS inserts,
       SUM(m.updates)                                      AS updates,
       SUM(m.deletes)                                      AS deletes,
       MAX(m.truncated)                                    AS truncated,
       MAX(m.timestamp)                                    AS
last_modification_flush,
       MAX(t.last_analyzed)                                AS last_analyzed
FROM   all_tab_modifications m
LEFT JOIN all_tables t ON t.owner = m.table_owner AND t.table_name =
m.table_name
WHERE  m.table_owner IN (&TARGET_SCHEMAS)
GROUP  BY m.table_owner, m.table_name
ORDER  BY SUM(m.inserts) + SUM(m.updates) + SUM(m.deletes) DESC;
SPOOL OFF

-- 11e  Database-side code and objects per layer: PL/SQL that ODI procedures
call, views,
--      materialized views, triggers. All of it has to move or be rewritten.
SPOOL odi_q11e_dwh_objects.csv
SELECT o.owner                                             AS schema_layer,
       o.object_type,
       COUNT(*)                                            AS object_cnt,
       COUNT(CASE WHEN o.status <> 'VALID' THEN 1 END)     AS invalid_cnt,
       (SELECT COUNT(*) FROM all_source s
        WHERE  s.owner = o.owner AND s.type = o.object_type) AS source_lines
FROM   all_objects o
WHERE  o.owner IN (&TARGET_SCHEMAS)
AND    o.object_type IN ('TABLE', 'VIEW', 'MATERIALIZED VIEW', 'PROCEDURE',
'FUNCTION', 'PACKAGE',
```

```
                          'PACKAGE BODY', 'TRIGGER', 'TYPE', 'SEQUENCE',
'SYNONYM')
GROUP  BY o.owner, o.object_type
ORDER  BY o.owner, o.object_type;
SPOOL OFF

-- 11f  Database links (names only), used by procedures flagged in 04.
SPOOL odi_q11f_db_links.csv
SELECT owner, db_link, username, created
FROM   all_db_links
ORDER  BY owner, db_link;
SPOOL OFF

--
================================================================================
=====
-- 12  SUMMARY COUNTS FOR THE ASSESSMENT     -> plan.xlsx "ODI Dagilimi",
assessment report
--     Run on the repository (back to the work schema).
--
================================================================================
=====
ALTER SESSION SET CURRENT_SCHEMA = &ODI_WORK_SCHEMA;

-- 12a  Packages per name prefix and effort group.
SPOOL odi_q12a_prefix_by_group.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
pkg AS (
    SELECT p.i_package,
           p.pack_name,
           pr.project_name,
           fld.folder_path,
           p.last_date,
           p.last_user
    FROM   snp_package p
    LEFT JOIN fld         ON fld.i_folder  = p.i_folder
    LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
```

```
            WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
            WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
            WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
            WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
            WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
            WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
            WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
            WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
            WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
            WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
            WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
            WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
            WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
            WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
            WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
            WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
            ELSE 'OTHER'
          END                          AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
SOURCE
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                     SELECT 1
                     FROM   snp_map_cp   cp
                     JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                     WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
```

```
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
),
pagg AS (
    SELECT i_package,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' THEN ds_name END)
AS src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND also_target = 0 THEN
ds_name END)  AS ext_src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'TARGET' THEN ds_name END)
AS tgt_cnt,
           -- Assumes the target DWH is Oracle (MACFit). Change the literal for
other estates.
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND NVL(technology, '?')
<> 'ORACLE'
                               THEN ds_name END)
AS src_non_oracle_cnt
    FROM   pds
    GROUP  BY i_package
),
pstep AS (
    -- Step mix per package. Classified by which object column is filled, which
is more
    -- stable across versions than STEP_TYPE codes.
    SELECT s.i_package,
           COUNT(*)
AS step_cnt,
           COUNT(DISTINCT s.i_mapping)
AS mapping_cnt,
           COUNT(CASE WHEN s.i_trt IS NOT NULL THEN 1 END)
AS procedure_step_cnt,
           COUNT(DISTINCT s.i_trt)
AS procedure_cnt,
           COUNT(CASE WHEN s.i_var IS NOT NULL THEN 1 END)
AS variable_step_cnt,
           COUNT(DISTINCT s.i_var)
AS variable_cnt,
           COUNT(CASE WHEN s.step_type IN ('SE', 'OE') THEN 1 END)
AS tool_step_cnt,   -- VERIFY: STEP_TYPE 'SE' = ODI tool (OdiStartScen,
OdiFileMove...), 'OE' = OS command
           COUNT(CASE WHEN s.i_mapping IS NULL AND s.i_trt IS NULL AND s.i_var
IS NULL
                      THEN 1 END)
AS other_step_cnt
    FROM   snp_step s
    GROUP  BY s.i_package
),
pcomp AS (
    SELECT pm.i_package,
           COUNT(CASE WHEN comp.comp_class IN ('JOIN', 'LOOKUP', 'AGGREGATE',
'SET', 'PIVOT', 'UNPIVOT',
                                               'SPLIT', 'SUBQUERY_FILTER',
```

```
'REUSABLE_MAPPING', 'TABLE_FUNCTION')
                      THEN 1 END)
AS heavy_comp_cnt,
           COUNT(CASE WHEN comp.comp_class NOT IN ('DATASTORE', 'SIGNATURE',
'DATASET') THEN 1 END) AS transform_comp_cnt
    FROM   pmap pm
    JOIN   comp ON comp.i_mapping = pm.i_mapping
    GROUP  BY pm.i_package
),
cls AS (
    -- Rev2 effort-group rule, evaluated in this order:
    --   1 mapping not visible   : package has no mapping step
    --   2 target unclear        : mapping steps exist but no target datastore
resolved
    --   3 5+ sources or multiple targets
    --   4 2-4 sources, single target
    --   5 single source, single target (also catches 0 sources + 1 target, e.g.
truncate/insert-only)
    SELECT pkg.i_package,
           pkg.pack_name,
           pkg.project_name,
           pkg.folder_path,
           REGEXP_SUBSTR(pkg.pack_name, '^[^_]+_[^_]+')            AS
name_prefix,   -- e.g. PKG_EXT, PCK_ODS
           NVL(ps.step_cnt, 0)            AS step_cnt,
           NVL(ps.mapping_cnt, 0)         AS mapping_cnt,
           NVL(ps.procedure_step_cnt, 0)  AS procedure_step_cnt,
           NVL(ps.procedure_cnt, 0)       AS procedure_cnt,
           NVL(ps.variable_step_cnt, 0)   AS variable_step_cnt,
           NVL(ps.tool_step_cnt, 0)       AS tool_step_cnt,
           NVL(ps.other_step_cnt, 0)      AS other_step_cnt,
           NVL(pa.src_cnt, 0)             AS src_cnt,
           NVL(pa.ext_src_cnt, 0)         AS ext_src_cnt,
           NVL(pa.tgt_cnt, 0)             AS tgt_cnt,
           NVL(pa.src_non_oracle_cnt, 0)  AS src_non_oracle_cnt,
           NVL(pc.heavy_comp_cnt, 0)      AS heavy_comp_cnt,
           NVL(pc.transform_comp_cnt, 0)  AS transform_comp_cnt,
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN '1
Mapping not visible'
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN '2
Target unclear'
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN '3 5+
sources or multiple targets'
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN '4 2-4
sources / single target'
             ELSE                                                          '5
Single source / single target'
           END                            AS effort_group,
           -- Rev2 unit hours (analysis + conversion + test). Assumptions,
calibrate after the pilot.
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN 5.5
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN 5.5
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN 8
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN 5.5
             ELSE                                                          3
           END                            AS rev2_unit_hours
    FROM   pkg
    LEFT JOIN pstep ps ON ps.i_package = pkg.i_package
    LEFT JOIN pagg  pa ON pa.i_package = pkg.i_package
    LEFT JOIN pcomp pc ON pc.i_package = pkg.i_package
)
SELECT NVL(cls.name_prefix, '(no prefix)')
```

```
AS name_prefix,
       COUNT(*)
AS packages,
       COUNT(CASE WHEN cls.effort_group LIKE '1%' THEN 1 END)
AS g1_mapping_not_visible,
       COUNT(CASE WHEN cls.effort_group LIKE '2%' THEN 1 END)
AS g2_target_unclear,
       COUNT(CASE WHEN cls.effort_group LIKE '3%' THEN 1 END)
AS g3_5plus_src_or_multi_tgt,
       COUNT(CASE WHEN cls.effort_group LIKE '4%' THEN 1 END)
AS g4_2to4_src,
       COUNT(CASE WHEN cls.effort_group LIKE '5%' THEN 1 END)
AS g5_single_src,
       COUNT(CASE WHEN cls.effort_group LIKE '5%' AND cls.heavy_comp_cnt = 0
                  AND cls.src_non_oracle_cnt > 0 THEN 1 END)
AS datastream_candidates_non_oracle_src,
       SUM(cls.procedure_cnt)
AS procedure_refs
FROM   cls
GROUP  BY NVL(cls.name_prefix, '(no prefix)')
ORDER  BY packages DESC;
SPOOL OFF

-- 12b  Effort groups with Rev2 hours. Person-days = hours / 8, plus 20%
reserve.
--      Compare with the Rev2 counts (283 / 120 / 69 / 121 / 121) before reusing
the plan.
SPOOL odi_q12b_effort_groups.csv
WITH
fld AS (
    -- Folder path inside each project (folders can be nested).
    SELECT f.i_folder,
           f.i_project,
           SUBSTR(SYS_CONNECT_BY_PATH(REPLACE(f.folder_name, '/', '-'), '/'), 2)
AS folder_path
    FROM   snp_folder f
    START WITH f.par_i_folder IS NULL          -- VERIFY: parent-folder column
PAR_I_FOLDER (alt: I_PAR_FOLDER)
    CONNECT BY PRIOR f.i_folder = f.par_i_folder
),
pkg AS (
    SELECT p.i_package,
           p.pack_name,
           pr.project_name,
           fld.folder_path,
           p.last_date,
           p.last_user
    FROM   snp_package p
    LEFT JOIN fld         ON fld.i_folder  = p.i_folder
    LEFT JOIN snp_project pr ON pr.i_project = fld.i_project
),
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
```

```
          CASE
            WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
            WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
            WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
            WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
            WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
            WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
            WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
            WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
            WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
            WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
            WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
            WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
            WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
            WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
            WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
            WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
            WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
            ELSE 'OTHER'
          END                          AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
          ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
SOURCE
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                    SELECT 1
                    FROM   snp_map_cp   cp
                    JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                    WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
```

```
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
),
pagg AS (
    SELECT i_package,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' THEN ds_name END)
AS src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND also_target = 0 THEN
ds_name END)  AS ext_src_cnt,
           COUNT(DISTINCT CASE WHEN ds_role = 'TARGET' THEN ds_name END)
AS tgt_cnt,
           -- Assumes the target DWH is Oracle (MACFit). Change the literal for
other estates.
           COUNT(DISTINCT CASE WHEN ds_role = 'SOURCE' AND NVL(technology, '?')
<> 'ORACLE'
                               THEN ds_name END)
AS src_non_oracle_cnt
    FROM   pds
    GROUP  BY i_package
),
pstep AS (
    -- Step mix per package. Classified by which object column is filled, which
is more
    -- stable across versions than STEP_TYPE codes.
    SELECT s.i_package,
           COUNT(*)
AS step_cnt,
           COUNT(DISTINCT s.i_mapping)
AS mapping_cnt,
           COUNT(CASE WHEN s.i_trt IS NOT NULL THEN 1 END)
AS procedure_step_cnt,
           COUNT(DISTINCT s.i_trt)
AS procedure_cnt,
           COUNT(CASE WHEN s.i_var IS NOT NULL THEN 1 END)
AS variable_step_cnt,
           COUNT(DISTINCT s.i_var)
AS variable_cnt,
           COUNT(CASE WHEN s.step_type IN ('SE', 'OE') THEN 1 END)
AS tool_step_cnt,   -- VERIFY: STEP_TYPE 'SE' = ODI tool (OdiStartScen,
OdiFileMove...), 'OE' = OS command
           COUNT(CASE WHEN s.i_mapping IS NULL AND s.i_trt IS NULL AND s.i_var
IS NULL
                      THEN 1 END)
AS other_step_cnt
    FROM   snp_step s
    GROUP  BY s.i_package
),
pcomp AS (
    SELECT pm.i_package,
           COUNT(CASE WHEN comp.comp_class IN ('JOIN', 'LOOKUP', 'AGGREGATE',
```

```
'SET', 'PIVOT', 'UNPIVOT',
                                                'SPLIT', 'SUBQUERY_FILTER',
'REUSABLE_MAPPING', 'TABLE_FUNCTION')
                      THEN 1 END)
AS heavy_comp_cnt,
           COUNT(CASE WHEN comp.comp_class NOT IN ('DATASTORE', 'SIGNATURE',
'DATASET') THEN 1 END) AS transform_comp_cnt
    FROM   pmap pm
    JOIN   comp ON comp.i_mapping = pm.i_mapping
    GROUP  BY pm.i_package
),
cls AS (
    -- Rev2 effort-group rule, evaluated in this order:
    --   1 mapping not visible   : package has no mapping step
    --   2 target unclear        : mapping steps exist but no target datastore
resolved
    --   3 5+ sources or multiple targets
    --   4 2-4 sources, single target
    --   5 single source, single target (also catches 0 sources + 1 target, e.g.
truncate/insert-only)
    SELECT pkg.i_package,
           pkg.pack_name,
           pkg.project_name,
           pkg.folder_path,
           REGEXP_SUBSTR(pkg.pack_name, '^[^_]+_[^_]+')            AS
name_prefix,   -- e.g. PKG_EXT, PCK_ODS
           NVL(ps.step_cnt, 0)           AS step_cnt,
           NVL(ps.mapping_cnt, 0)        AS mapping_cnt,
           NVL(ps.procedure_step_cnt, 0) AS procedure_step_cnt,
           NVL(ps.procedure_cnt, 0)      AS procedure_cnt,
           NVL(ps.variable_step_cnt, 0)  AS variable_step_cnt,
           NVL(ps.tool_step_cnt, 0)      AS tool_step_cnt,
           NVL(ps.other_step_cnt, 0)     AS other_step_cnt,
           NVL(pa.src_cnt, 0)            AS src_cnt,
           NVL(pa.ext_src_cnt, 0)        AS ext_src_cnt,
           NVL(pa.tgt_cnt, 0)            AS tgt_cnt,
           NVL(pa.src_non_oracle_cnt, 0) AS src_non_oracle_cnt,
           NVL(pc.heavy_comp_cnt, 0)     AS heavy_comp_cnt,
           NVL(pc.transform_comp_cnt, 0) AS transform_comp_cnt,
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN '1
Mapping not visible'
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN '2
Target unclear'
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN '3 5+
sources or multiple targets'
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN '4 2-4
sources / single target'
             ELSE                                                          '5
Single source / single target'
           END                           AS effort_group,
           -- Rev2 unit hours (analysis + conversion + test). Assumptions,
calibrate after the pilot.
           CASE
             WHEN NVL(ps.mapping_cnt, 0) = 0                          THEN 5.5
             WHEN NVL(pa.tgt_cnt, 0) = 0                              THEN 5.5
             WHEN NVL(pa.src_cnt, 0) >= 5 OR pa.tgt_cnt > 1           THEN 8
             WHEN pa.src_cnt BETWEEN 2 AND 4                          THEN 5.5
             ELSE                                                          3
           END                           AS rev2_unit_hours
    FROM   pkg
    LEFT JOIN pstep ps ON ps.i_package = pkg.i_package
    LEFT JOIN pagg  pa ON pa.i_package = pkg.i_package
    LEFT JOIN pcomp pc ON pc.i_package = pkg.i_package
```

```
)
SELECT cls.effort_group,
       COUNT(*)                                                AS packages,
       MAX(cls.rev2_unit_hours)                                AS unit_hours,
       SUM(cls.rev2_unit_hours)                                AS total_hours,
       ROUND(SUM(cls.rev2_unit_hours) / 8, 1)                  AS person_days,
       ROUND(SUM(cls.rev2_unit_hours) / 8 * 1.2, 1)            AS
person_days_with_20pct_reserve,
       ROUND(100 * RATIO_TO_REPORT(COUNT(*)) OVER (), 1)       AS
pct_of_packages
FROM   cls
GROUP  BY cls.effort_group
ORDER  BY cls.effort_group;
SPOOL OFF

-- 12c  Source technologies: datastores, logical schemas (proxy for source
systems), packages.
SPOOL odi_q12c_source_technologies.csv
WITH
comp AS (
    -- One row per logical mapping component, with a normalised class.
    -- LIKE patterns are used because component type names differ slightly
between patch levels.
    SELECT c.i_map_comp,
           c.i_owner_mapping           AS i_mapping,   -- VERIFY: owner column
I_OWNER_MAPPING (alt: I_MAPPING)
           c.name                      AS comp_name,
           c.i_map_ref,                                -- VERIFY:
SNP_MAP_COMP.I_MAP_REF points to the bound object (datastore / reusable mapping)
           UPPER(ct.type_name)         AS comp_type,   -- VERIFY:
SNP_MAP_COMP_TYPE.TYPE_NAME (alt: INTERNAL_NAME, NAME), if no type table,
SNP_MAP_COMP.TYPE_NAME
           CASE
             WHEN UPPER(ct.type_name) LIKE '%DATASTORE%'      THEN 'DATASTORE'
             WHEN UPPER(ct.type_name) LIKE '%JOIN%'           THEN 'JOIN'
             WHEN UPPER(ct.type_name) LIKE '%SUBQUERY%'       THEN
'SUBQUERY_FILTER'
             WHEN UPPER(ct.type_name) LIKE '%FILTER%'         THEN 'FILTER'
             WHEN UPPER(ct.type_name) LIKE '%EXPRESSION%'     THEN 'EXPRESSION'
             WHEN UPPER(ct.type_name) LIKE '%AGGREGAT%'       THEN 'AGGREGATE'
             WHEN UPPER(ct.type_name) LIKE '%LOOKUP%'         THEN 'LOOKUP'
             WHEN UPPER(ct.type_name) LIKE 'SET%'             THEN 'SET'
             WHEN UPPER(ct.type_name) LIKE '%SORT%'           THEN 'SORT'
             WHEN UPPER(ct.type_name) LIKE '%SPLIT%'          THEN 'SPLIT'
             WHEN UPPER(ct.type_name) LIKE '%DISTINCT%'       THEN 'DISTINCT'
             WHEN UPPER(ct.type_name) LIKE '%UNPIVOT%'        THEN 'UNPIVOT'
             WHEN UPPER(ct.type_name) LIKE '%PIVOT%'          THEN 'PIVOT'
             WHEN UPPER(ct.type_name) LIKE '%REUSABLE%'       THEN
'REUSABLE_MAPPING'
             WHEN UPPER(ct.type_name) LIKE '%TABLE%FUNCTION%' THEN
'TABLE_FUNCTION'
             WHEN UPPER(ct.type_name) LIKE '%DATASET%'        THEN 'DATASET'
             WHEN UPPER(ct.type_name) LIKE '%SIGNATURE%'      THEN 'SIGNATURE'
             ELSE 'OTHER'
           END                         AS comp_class
    FROM   snp_map_comp c
    LEFT JOIN snp_map_comp_type ct
           ON ct.i_map_comp_type = c.i_map_comp_type   -- VERIFY: join column
I_MAP_COMP_TYPE on both tables
),
ds AS (
    -- Datastore components with their role. Rule: a datastore component that
receives an
    -- incoming connection is a TARGET, one with no incoming connection is a
```

```
SOURCE
    -- (lookup sources count as sources).
    SELECT comp.i_mapping,
           comp.i_map_comp,
           CASE WHEN EXISTS (
                    SELECT 1
                    FROM   snp_map_cp   cp
                    JOIN   snp_map_conn cn ON cn.i_end_map_cp = cp.i_map_cp
-- VERIFY: SNP_MAP_CONN.I_END_MAP_CP (alt: I_TO_MAP_CP), SNP_MAP_CP.I_MAP_CP
                    WHERE  cp.i_owner_map_comp = comp.i_map_comp)
-- VERIFY: SNP_MAP_CP.I_OWNER_MAP_COMP
                THEN 'TARGET' ELSE 'SOURCE' END          AS ds_role,
           CASE WHEN t.i_table IS NOT NULL
                THEN md.cod_mod || '.' || t.table_name   -- model code +
datastore name (form sheet 3 format, e.g. MDL_ODS.T_ODS_X)
                ELSE NVL(r.qualified_name, comp.comp_name)
           END                                           AS ds_name,
           md.tech_int_name                              AS technology,   --
VERIFY: SNP_MODEL.TECH_INT_NAME (alt: join SNP_TECHNO via I_TECHNO)
           md.lschema_name                               AS lschema_name  --
VERIFY: SNP_MODEL.LSCHEMA_NAME
    FROM   comp
    LEFT JOIN snp_map_ref r ON r.i_map_ref = comp.i_map_ref
    LEFT JOIN snp_table   t ON t.i_table   = r.i_ref_id     -- VERIFY:
SNP_MAP_REF.I_REF_ID holds SNP_TABLE.I_TABLE for datastore refs (check
ADAPTER_INTF_TYPE in 00d)
    LEFT JOIN snp_model  md ON md.i_mod    = t.i_mod
    WHERE  comp.comp_class = 'DATASTORE'
),
pmap AS (
    -- Mappings called from package steps.
    SELECT DISTINCT s.i_package, s.i_mapping
    FROM   snp_step s
    WHERE  s.i_mapping IS NOT NULL      -- VERIFY: 12c column SNP_STEP.I_MAPPING
(11g interfaces used I_POP, upgraded repos convert them to mappings)
),
pds AS (
    -- Distinct datastores per package and role. also_target = 1 when a "source"
is also
    -- written by another mapping of the same package (intermediate / staging
table).
    SELECT x.*,
           MAX(CASE WHEN x.ds_role = 'TARGET' THEN 1 ELSE 0 END)
               OVER (PARTITION BY x.i_package, x.ds_name) AS also_target
    FROM  (SELECT DISTINCT pm.i_package, ds.ds_role, ds.ds_name, ds.technology,
ds.lschema_name
           FROM   pmap pm
           JOIN   ds ON ds.i_mapping = pm.i_mapping) x
)
SELECT NVL(technology, '(unresolved)')                          AS technology,
       COUNT(DISTINCT CASE WHEN also_target = 0 THEN ds_name END)  AS
source_datastores,
       COUNT(DISTINCT lschema_name)                             AS
logical_schemas,
       COUNT(DISTINCT i_package)                                AS packages
FROM   pds
WHERE  ds_role = 'SOURCE'
GROUP  BY NVL(technology, '(unresolved)')
ORDER  BY source_datastores DESC;
SPOOL OFF

-- 12d  Object counts for the estate (one row).
SPOOL odi_q12d_object_counts.csv
SELECT (SELECT COUNT(*) FROM snp_project)                                AS
```

```
projects,
       (SELECT COUNT(*) FROM snp_package)                                    AS
packages,
       (SELECT COUNT(*) FROM snp_mapping)                                    AS
mappings,
       (SELECT COUNT(DISTINCT i_mapping) FROM snp_step WHERE i_mapping IS NOT
NULL) AS mappings_used_in_packages,
       (SELECT COUNT(DISTINCT i_trt) FROM snp_step WHERE i_trt IS NOT NULL)  AS
procedures_used_in_packages,
       (SELECT COUNT(*) FROM snp_var)                                        AS
variables,
       (SELECT COUNT(*) FROM snp_model)                                      AS
models,
       (SELECT COUNT(*) FROM snp_table)                                      AS
datastores,
       (SELECT COUNT(*) FROM snp_scen)                                       AS
scenario_versions,
       (SELECT COUNT(DISTINCT scen_name) FROM snp_scen)                      AS
scenarios,
       (SELECT COUNT(*) FROM snp_load_plan)                                  AS
load_plans,
       (SELECT COUNT(*) FROM snp_plan_agent)                                 AS
schedules,
       (SELECT COUNT(*) FROM snp_session
        WHERE  sess_beg >= TRUNC(SYSDATE) - &LOOKBACK_DAYS)                  AS
sessions_in_window,
       (SELECT MIN(sess_beg) FROM snp_session)                               AS
oldest_session_in_log
FROM   dual;
SPOOL OFF

-- End of script.
```